

After Chat: The Three Transitions Between Non-Programmers and Agentic AI

The terminal is no longer a terminal: a four-level model, the evidence of 2025 and 2026, and a preregistered programme to measure the crossings

Vadym Chernets

September 2026

Vadym Chernets, PhD, AI systems architect · ORCID [0009-0007-4845-3163](https://orcid.org/0009-0007-4845-3163)

Download this paper

- **SSRN** (version of record): ssrn.com/abstract=7520019 · DOI [10.2139/ssrn.7520019](https://doi.org/10.2139/ssrn.7520019)
- **Zenodo** (archived, open access): doi.org/10.5281/zenodo.22943313
- **GitHub** (full text as Markdown and PDF):
github.com/vadimchernets/papers/tree/main/after-chat; code:
github.com/vadimchernets/after-chat

The same text is in three places so that it can be reached when one of them cannot serve it. SSRN holds the version of record; cite that one.

Keywords: agentic AI; AI agents; non-programmers; end-user programming; AI literacy; knowledge workers; technology adoption; generative AI adoption; human oversight; delegation; second-level digital divide; correlated errors; ironies of automation; deskilling; preregistration

Abstract

Most professionals meet AI as a chatbot, while the same models, as AI agents, now act on files and programs. I ask why so few non-programmers have crossed from chat to agentic AI, and how to measure the crossing. The answer rests on a four-level model defined by reach and role and independent of products: chat, a vendor-bounded safe agentic environment, an agent on one's own computer, and repeatable work orchestrated across several models. A transition counts only when an agent has produced an artefact the person asked for. Installing software counts as an attempt. Evidence from 2025 to September 2026 indicates that the barrier to installation and to a first bounded attempt has fallen, while reliability on whole pieces of work, product stability and folder safety have not. Installation is one step, the person's own chatbot can guide it ("reflexive onboarding"), and two vendors merged agent and chat surfaces in 2026. One vendor's press-reported,

directional figure puts knowledge workers at a fifth of its coding agent's five million weekly users; it measures use of a terminal-heritage agent, not level-3 adoption as defined here. Use remains broad and shallow: about half of US workers use AI, about one in seven daily, and few for automation. The barrier has moved from installing to directing. Drawing on end-user programming research, I name the missing capability workflow vocabulary and set out the manager's work as a delegation contract: goal, boundary, artefact and check. Web orchestration (one question to several chatbots) is proposed as the bridge that teaches comparison, and CLI orchestration (one agent calling others) as the durable layer, bounded by the fact that correlated models' agreement is weak evidence. The paper closes with fifteen falsifiable propositions, the design of a preregistered cohort programme without its results, and open teaching code.

Executive summary

The terminal, the black window that stood for everything non-programmers could not do with AI, has changed its meaning. It is now the place on a person's own machine where an agent works, and the skill it demands is no longer typing commands. The skill is managing the worker that types them.

Professional developers are furthest along: 90% of more than 15,000 surveyed in 2026 used agentic tools at least weekly (a vendor survey), although weekly use is not yet the manager's competence of Section 8. Everyone else is mostly still talking to a chatbot. 52% of US workers use AI in their role, 15% daily, and 16% of those users name automation among their uses.

The reason is no longer that agents are hard to install. Installing one takes a command or a click, the chatbot a person already uses can talk them through it, and the vendors have begun merging chat with agents. What stays hard is different in kind. Agents still fail on whole pieces of work, the product a person learns may be renamed or retired within a year, and a granted folder still needs guarding. After the installer, the first thing that stops people is that they do not know what to hand over or how to check it, the gap that Section 1 names workflow vocabulary. Behind it come the first error, permissions and safety, organisational rules, hardware and cost, and the competence to judge results.

The model has four levels, ordered by how far the model reaches and by what the person must do: conversation, a fenced space the vendor runs, the person's own computer, and work directed across several models. People may skip a level. A crossing is shown only by an artefact the agent produced, so that installations, badges and self-report do not count.

Orchestration, sending one question to several models, comes in two layers. In the browser it costs nothing to try and teaches the habit of comparing answers, and on the command line it can be written down and run again. In both, what counts is the number of model families asked, because correlated errors cap what agreement is worth.

Managing agents is a competence of its own. It has a delegation contract in four parts (goal, boundary, artefact, check), four practices (delegate, supervise, verify, revoke), a stopping

rule for reviews, and an acceptance record that separates what the agent produced from what the person relied on.

The argument is built to be tested. Fifteen propositions carry measures and falsifiers. The design of a preregistered cohort programme (OSF DOI 10.17605/OSF.IO/X4EGQ) is described here, and its results will be reported separately. A status table says how firmly each claim is held, and open code is in Appendix C.

1. Introduction

Consider an office professional in September 2026. She uses a chatbot every week. She asks it to draft an email, summarise a report, suggest a formula. What she receives is text, and she carries that text by hand into the place where her work lives: a folder of contracts, a spreadsheet of accounts, a slide deck due on Friday. The same model, through a different door, can open that folder, read every file in it, write the spreadsheet, build the deck and check its own arithmetic. Most professionals, as far as the surveys of Section 4.4 can tell, have not yet used that door, and many do not know that it exists.

For two years the door looked like a terminal: a black window, a blinking cursor, a command to paste. That picture is out of date. The terminal stayed a terminal only visually. It became the environment in which an agent lives, the place on a person's own machine where a model can read, write, run and schedule, and the person in front of it no longer needs to operate a shell so much as direct the agent that does. What changes is the human role, from someone who asks a model questions to someone who manages a worker.

My subject is the road between those two roles and who is on it. Among professional developers the tools, at least, have already arrived. A survey of more than 15,000 professional developers in May to July 2026 found that 90% used AI coding agents at work at least weekly and 68% daily, and that the share using one terminal agent, Claude Code, at work had risen from about 3% in the spring of 2025 to 18% in January 2026 and 39% by mid-2026 (JetBrains Research, 2026a, 2026b; a vendor survey, since JetBrains sells a coding agent). Among everyone else it has not, and the pattern of use is broad and shallow. In the same months, 52% of US workers reported using AI in their role, 15% used it daily, and only 16% of those users named automation among their uses; writing and search dominated (Gallup, 2026). For the typical professional AI is still a conversation, and the St. Louis Fed's figures say the same over a longer span: between August 2024 and May 2026 the share of workers using generative AI for their jobs rose from 33% to 45%, while fewer than 3% of tasks had adoption rates above 50% (Bick et al., 2026).

The standard explanation of the gap is technical difficulty: agents are for people who can program. That answer was reasonable in 2024 and is weak in 2026. Installing a terminal agent now takes one command or a desktop installer, and the leading agent ships a desktop application whose documentation says plainly, "No terminal required" (Anthropic, 2026a). The vendors have gone further. OpenAI turned its coding-agent application into the default ChatGPT desktop client on 9 July 2026 and added an agent, ChatGPT Work, that

uses local files and applications (MacRumors, 2026). Anthropic merged its desktop agent for knowledge work, Cowork, into the ordinary chat on 16 September 2026; TechCrunch reported the company as saying that customers “often struggled to choose the right tab for the right task” (TechCrunch, 2026a). OpenAI reported, through the press, that knowledge workers made up about 20% of the five million weekly users of its coding agent in June 2026 and were adopting it more than three times as fast as developers (Help Net Security, 2026). The figure is vendor-reported and directional, the vendor’s own page was not accessible to my checks, and it measures knowledge-worker use of a terminal-heritage agent, which is wider than level-3 adoption as defined in Section 3. It is hard to see where the technical barrier is in that picture.

The barrier has moved. It has moved from installing to directing: from “how do I get this running?” to “what do I ask it to do, inside what limits, and how will I know it did it?” The second question is harder than it sounds, and the literature on end-user programming explains why. Non-experts struggle to express intent to language models even when access is free and immediate (Zamfirescu-Pereira et al., 2023), and they must learn which phrasings map onto what the system can do (Liu et al., 2023). Delegating to an AI also shifts the work from producing to specifying and checking (Sarkar & Drosos, 2025; Simkute et al., 2024). I call the missing capability *workflow vocabulary*: a working stock of tasks a person knows how to hand to an agent, phrased so that the agent can carry them out and the result can be checked. Stated formally, it is the mapping between a person’s domain goals and the sub-tasks an agent can execute, with the words for inputs, transformations, outputs, exceptions and evidence that the mapping needs. A developer who opens an agent already knows the first ten things to type. An accountant or a teacher usually does not, and nobody in their office has told them.

The first contribution is a four-level model of the road from chat to agentic work, defined by reach and role and by no product name. The levels run from chat through a safe agentic environment and an agent on one’s own computer to repeatable orchestrated work. Each is defined by what the agent can touch and by what the person must specify and check, and each transition is defined by an artefact the agent produced, and installed software alone does not count. Because the definitions do not depend on product names, the model survives the vendors’ current habit of merging their products (Section 3).

The second is an evidence synthesis for 2025 to September 2026. It shows that installation and a first bounded attempt have become much easier, that the three limits set out in Section 4.5 (reliable work on whole tasks, stable products, safe folders) have not moved with them, and that the shallow pattern of use persists. It also locates the barriers that remain: workflow vocabulary, the error that stops a newcomer, permission and safety, organisational rules, and the competence to judge results (Sections 4 and 5).

The third is an account of the two orchestration layers. Web orchestration, one question sent to several consumer chatbots, is the bridge: it needs no installation and teaches the habit of comparison. CLI orchestration, one agent calling other agents’ command-line tools, is the layer of delegation that lasts. Both are bounded by a fact that users tend to

miss, that agreement among models is not independent evidence (Chernets, 2026a) (Sections 6 and 7).

The fourth is a measurement programme with falsifiable propositions: the design of a preregistered cohort programme that follows non-programmers across these transitions (Chernets, 2026d), and an open teaching toolkit of orchestration and safety patterns in the appendices (Sections 9 and 10, Appendix C).

A few limits of scope apply throughout. “Non-programmer” means a professional whose work is substantially at a computer and who has no training in scripting or software engineering. An analyst fluent in spreadsheet formulas counts, a developer does not, and developers appear here as a comparison group. I report no results from the cohort programme; those belong to a separate paper after its observation period. Most adoption figures come from the United States and from vendors, and are marked as such. And the tools described change by the month, so the model is built to outlast them and the product details are expected to date.

Section 2 places the argument in the literature and Section 3 defines the levels and transitions. Section 4 shows where the technical barrier went, Section 5 what replaced it, Sections 6 and 7 the two forms of orchestration, and Section 8 what the move does to the human role. Section 9 describes the programme that measures the crossings, Section 10 states the propositions, Section 11 draws implications, Section 12 sets boundary conditions, and Section 13 concludes.

2. Background: from end-user programming to end-user delegation

2.1 End users have always programmed

The idea that people who are not programmers direct computation is older than language models. Nardi (1993) argued that end users command computing power through task-specific languages and frameworks, spreadsheets being the canonical case, without needing general programming skill. Ko et al. (2011) showed that end users who build software face the same problems as professionals (requirements, reuse, testing, debugging) without the training, and named the field end-user software engineering. The wider field, end-user development, studies how people who are not professional developers create and adapt software for their own work. Myers, Pane and Ko (2004) spent years designing notations closer to how non-programmers describe tasks. Each of these lines of work assumed that the end user would still write something the machine executes, even if that something was a formula or a rule.

Language models changed what the end user writes. Sarkar et al. (2022) described programming with a large language model as a new activity, distinct from compilation, pair programming and search. Zamfirescu-Pereira et al. (2023) found that people without AI expertise struggle to design prompts that work reliably, overgeneralise from single successes and fall back on human conversational habits. Liu et al. (2023) named the difficulty of learning which words produce which behaviour “grounded abstraction

matching”. Sarkar and Drosos (2025), in the first empirical study of “vibe coding”, found that conversational programming redistributes expertise toward managing context and verifying results quickly, and leaves the need for judgement where it was. Virk and Liu (2025) found that business users given AI-written analysis code often failed to catch critical errors even when told to look for them. Feldman and Anderson (2024) called non-expert programmers the most under-studied population of the generative era.

I take the next step in that line. The unit of end-user work has changed from a formula, a script or a prompt to a delegation: a task handed to an agent that acts on files and tools over many steps. The literature on end-user programming explains why expressing intent is hard, and the literature on delegation and oversight explains why checking the result is hard. The four-level model of Section 3 is built to join the two.

2.2 Adoption is fast and shallow

Generative AI spread faster than the personal computer or the internet. Bick, Blandin and Deming (2024) estimated that about 40% of US working-age adults used it by late 2024, while only a small share of work hours was assisted. Chatterji et al. (2025) found that by mid-2025 ChatGPT served about 700 million weekly users and that most messages were not work-related. In Denmark, Humlum and Vestergaard (2025) found precise null effects on earnings and hours two years after adoption, and a gap in use between women and men; a meta-analysis across more than 100 countries puts that gap at about 16 percentage points since early 2025 (Cranney et al., 2026). Across 35 European countries, exposure to AI did not translate automatically into use, and training, digitalisation and worker agency mattered more (Henseke, 2026).

Field experiments show large gains where the tool fits the task: 14% more customer issues resolved per hour on average and 34% for novices (Brynjolfsson et al., 2025), faster and better writing in a trial with 444 professionals, with the largest gains for the weaker writers (Noy & Zhang, 2023), and, among 758 consultants, 12.2% more tasks completed and quality up by more than 40% inside the “jagged frontier” of model capability, with worse work on a task outside it (Dell’Acqua et al., 2026). A randomised trial with experienced open-source developers found the opposite sign in early 2025, a 19% slowdown (Becker et al., 2025), a result its authors later said was likely out of date as tools improved (METR, 2026). The lesson of these trials is that gains depend on the fit between the task, the tool and the person’s ability to direct and check it, and those are the three things that a transition between levels changes.

2.3 Acceptance, diffusion and the second-level divide

Research on technology adoption, from the technology acceptance model (Davis, 1989) to the unified theory of acceptance and use of technology (Venkatesh et al., 2003), holds that perceived usefulness and ease of use gate adoption, and that social influence and facilitating conditions matter as much as individual attitudes. Diffusion research describes adoption as a social process with early and late adopters (Rogers, 2003). Once basic access to the internet became common, Hargittai (2002) showed that the meaningful

inequality moved to skill: the second-level digital divide. Van Dijk (2020) extended this to a sequence of motivation, access, skills and usage.

The same shift is visible now. Access to chat is nearly universal among office workers in rich countries, while the skill of directing an agent is far from universal. An AI divide has already been measured along the old lines of education, region and age (Daepf & Counts, 2024; Pew Research Center, 2026). What I add is the observation that the divide now runs *inside* the population of AI users, between those who converse with models and those who delegate to them.

2.4 Automation, oversight and the manager's burden

The literature on human oversight of automation begins with an irony. Bainbridge (1983) described it: automating most of a task leaves the human the hardest residue, monitoring and rare intervention, while eroding the skill needed for it. Parasuraman and Riley (1997) distinguished misuse, disuse and abuse of automation, and Lee and See (2004) showed that reliance should follow calibrated trust. The generative era has renewed each point. Knowledge workers who trust the tool more report less critical thinking (Lee et al., 2025). Generative AI shifts people from producing to evaluating and makes easy tasks easier and hard tasks harder (Simkute et al., 2024). Interventions to calibrate reliance on language models reduce over-reliance without producing appropriate reliance (Bo et al., 2025). Users of AI agents as daily assistants miscalibrate trust toward plausible but wrong plans, and do better when they take part in step-wise execution (He et al., 2025). Better traces of what an agent did speed up the detection of errors without reliably improving final accuracy (Grunde-McLaughlin et al., 2026). A study of about 73,000 public posts on one autonomous agent found that users' values were mostly met in what the agent delivered and mostly unmet in how they could supervise it (Ma et al., 2026). In a small laboratory study and three deployments, comfort with delegation grew over time while people still wanted control at moments of uncertainty (Long et al., 2025). Tomašev, Franklin and Osindero (2026) treat delegation to AI as a framework of task allocation, transfer of authority, accountability and trust, applicable to human and machine delegates alike.

On the cognitive side, an EEG study of 54 essay writers reported weaker connectivity and lower ownership in the assisted group, a "cognitive debt" (Kosmyna et al., 2025), and a published comment contested its methods (Stankovic et al., 2026). The safe reading is that offloading routine steps frees attention and that the freed attention has to be spent on checking. Two further ideas from learning research are used later. Cognitive load theory explains why a newcomer dropped into an unfenced environment stalls before a mental schema has formed (Sweller, 1988). And a model of adoption as Bayesian learning shows how people can settle into a self-reinforcing low-use equilibrium that only a first success or training can shift (Ma et al., 2025).

Two companion papers in this series bear on the same problem from the other side. The first shows that agreement among several models is weaker evidence than it looks, because their errors are correlated (Chernets, 2026a). The second shows how automated

oversight components can widen their own mandate and obstruct the work they guard (Chernets, 2026b). Both matter for a newcomer: the first for what orchestration can buy, the second for why safety layers can stop a person at the door.

2.5 What has not been studied

A search run on 21 September 2026 (web search, Crossref, arXiv and Semantic Scholar, with attention to CHI, UIST and VL/HCC papers of 2024 to 2026) found no published field study of non-programmers adopting terminal or command-line agents by name, no study of web orchestration (one question to several consumer chatbots in the browser) as a named practice, no treatment of managing agents as a skill with its own competency structure, and no study of an AI system scaffolding a person's adoption of agentic AI itself; the closest work applies scaffolding theory to AI tutors of other subjects (Cohn et al., 2026) or to task-conditioned interfaces for complex software (Liu & Sra, 2026). The labour-economics work measures generative AI adoption in general, and none of it isolates the step from chat to agentic tools among non-programmers. These gaps define the space the model maps and the programme in Section 9 is built to measure.

3. Four levels and three transitions

3.1 Two axes: reach and role

A level describes a relation between a person and a model at work, and I define it on two axes, reach and role.

Reach is what the model can touch on the person's behalf. At the narrowest it is nothing but the conversation, then files and tools inside a space the vendor bounds, then the person's own computer with its files, programs and schedule, and at the widest several models and agents at once.

Role is what the person must do for the work to succeed. In the same order, they ask and copy, they hand over a task and review the result, they own an agent's environment with its permissions and its undo, or they allocate work among agents and reconcile what comes back.

Products are deliberately absent from the definition. A single product can span several levels, and in 2026 the vendors are folding chat and agents into one window faster than any taxonomy built on product names could follow (Section 4.3). A level describes a piece of work, and one person, like one product, can span several. The same person can be at level 1 for one task in the morning and at level 3 for another in the afternoon.

The two axes also separate this model from the familiar scales of automation and autonomy. Parasuraman, Sheridan and Wickens (2000) classified how much of each stage of a task (acquiring information, analysing it, deciding, acting) a machine performs. Morris et al. (2024) graded general AI systems by the depth and breadth of their performance. Feng, McDonald and Zhang (2025) defined five levels of autonomy for AI agents by the role the user plays, from operator and collaborator to consultant, approver and observer.

Those scales describe how much an agent decides *within* a task. The four levels here describe *where* the work runs and *what the person owns*: a conversation, a vendor's fenced space, their own machine, a team of agents from several vendors. The axes are orthogonal. An agent on a person's own computer can run at any of Feng et al.'s levels of autonomy, from step-by-step approval to near-total delegation, and the person still owns its permissions and its undo. What the four levels add is a description of adoption: the order in which reach and responsibility grow for someone who starts at chat, and the evidence (an artefact) that shows each step was taken.

3.2 The four levels

Level 1: chat. The model's reach ends at the conversation. It produces text, code or images that the person copies into the place where work happens, and the person's role is to ask well and to carry the output. Almost every professional who uses AI works here most of the time (Section 2.2).

Level 2: a safe agentic environment. The model acts on files, connected services and sometimes a browser, inside a space the vendor bounds: selected folders, a virtual machine or server-side sandbox, connectors the person has approved, and an approval step before consequential actions. Anthropic's Cowork, OpenAI's ChatGPT Work, Microsoft's Copilot Cowork and Google's Gemini Spark are 2026 examples, and they share the design of a delegation agent working in a fenced space (Anthropic, 2026b; MacRumors, 2026; Microsoft, 2026a; TechCrunch, 2026b). The person's role changes from asking to delegating. They hand over a task that ends in a file or an action and review what comes back, and much of the environment's safety, at this level, is the vendor's responsibility.

Level 3: an agent on one's own computer. The agent runs on the person's machine with access to the file system, the shell, installed programs and scheduled jobs, through a terminal or a desktop shell over the same engine: Claude Code, OpenAI's Codex, Google's Antigravity CLI, xAI's Grok Build, Moonshot's Kimi Code and open-source equivalents. In the research literature these are LLM-based agents, and their use by developers is studied as agentic coding; the level asks what happens when the person running them is not a developer. The reach is the widest a single agent gets, and with it the person's role changes again. They own the environment: which folders the agent sees, which commands need approval, what is kept out, and how to undo a mistake. The terminal itself is incidental, since what defines the level is ownership of a working environment in which an agent can do anything the person could do. Cloud-hosted variants (the same agent loop run on a vendor's machine, with the person seeing a task and a result and never a prompt) are level 3 by capability and level 2 by blast radius, and the model records them as a variant of level 3 without forcing them into one cell.

Level 4: repeatable, orchestrated work. The person directs several models or agents, in parallel or in sequence, on work that recurs, and the reach now spans vendors. There are two layers. In *web orchestration* one question goes to several consumer chatbots in the browser, by hand or with a tool, and the answers are compared. In *CLI orchestration* one

agent calls other agents' command-line tools, collects their outputs and merges or checks them. The person's role is that of a manager: they set the goal, allocate the work, decide what counts as done, and reconcile disagreement. The work becomes repeatable when it is written down in a form the agents can run again, as an instruction file, a skill or a scheduled job. "Repeatable" describes the workflow, not the output. The same instruction file run next week will produce a different draft, and what repeats is the sequence of inputs, steps, checks and deliverable. A recurring single-agent job written down this way is level-3 work made repeatable, and it becomes level 4 when a second model family is directed and the results reconciled (transition 3, Section 3.3).

Table 1. *The four levels.*

Level	Reach of the model	Role of the person	Typical surfaces, September 2026	Crossing is shown by	What the level does not establish	Main barrier at entry
1. Chat	The conversation	Asks, copies output by hand	Chat in browser or app	(entry level)	That the model can act on the person's work, or be trusted without checking	None beyond access
2. Safe agentic environment	Files, connectors and browser inside a vendor-bounded space	Delegates a task, reviews the result	Cowork merged into Claude chat; ChatGPT Work; Copilot Cowork; Gemini Spark	A file or action produced by the agent from a delegated task, with an approval or refusal recorded	That the person can run an environment of their own, or repeat the work	Knowing what to delegate; trust
3. Agent on one's own computer	The person's file system, shell, programs, schedule	Owns the environment: permissions, exclusions, undo	Terminal and desktop shells of Claude Code, Codex, Antigravity CLI, Grok Build, Kimi Code	An artefact the agent produced on the person's own machine, in a folder the person set up, from a task needing more than one tool use	That the result is right, or safe to accept without review	Installation friction (falling); the first error; permissions; what to ask

Level	Reach of the model	Role of the person	Typical surfaces, September 2026	Crossing is shown by	What the level does not establish	Main barrier at entry
4. Orchestrated work	Several models or agents across vendors	Manages: goal, allocation, acceptance, reconciliation	Web: several chat tabs or broadcast tools. CLI: plugins and scripts that call other agents	A merged or cross-checked result from at least two model families, rerun at least once, with who was asked and who answered recorded	That agreement among the models is independent evidence, or that the task belongs at this level	Cost of reconciling; correlated errors; quota and tool churn

Two terms in the table are narrower than everyday usage. “Safe” in “safe agentic environment” names a design goal and an interaction pattern and gives no assurance. Folder scope, approvals, sandboxes and preview panes shrink the area on which an agent acts; they cannot make a buggy implementation preserve files or make a document carrying hidden instructions harmless, and Section 5.3 reports what has gone wrong inside granted folders. What the middle rung does establish is a practical mental model. A person who sees that the agent can write a report in one folder and cannot touch anything else has learned that delegated action has a perimeter, which is more useful than any abstract warning to be careful with AI. “Artefact” means a deliverable that needed several steps of the agent’s own work. The person who asked for it decides whether it counts as an artefact, a third party rates its quality (Section 9.4), and a study reports agreement between raters for both that rating and the coding of stop reasons. A folder with two empty files is a trace of an installer, and does not count.

3.3 Three transitions, and why an artefact defines each

A transition is crossed when the agent has produced, at the new level, an artefact the person asked for. An installation counts only as an attempt, and so do a completed tutorial, a badge, a licence and the self-report “I use agents now”. The rule has several consequences beyond the obvious one. A folder created by an agent shows that the agent ran, while a set of files written by the agent from the person’s description of their own work shows that the person delegated. The rule also makes the transitions observable from outside: a researcher or a manager can see an artefact and cannot see an intention or a feeling of competence. It makes failure informative, since a person who installed a tool and produced nothing has met a barrier worth naming, and counting the installation as success would hide it. And because it names no product, the rule lets studies compare across tools that are renamed every quarter.

Transition 1, from chat to a safe agentic environment, is crossed when a person first delegates a task that ends in a file or an action instead of in text. In 2026 this transition is being absorbed into the chat window itself. When the vendor’s model decides whether to answer or to act, as it does in the merged Claude experience announced on 16 September

2026 (Anthropic, 2026c), a person can cross it without deciding to. That makes the crossing easier and also less visible, including to the person.

Transition 2, from the safe environment to one's own computer, is crossed when a person runs an agent in their own environment and receives an artefact from it. The decisive change is ownership. At level 2 the vendor decides what the agent may touch, and at level 3 the person does, whatever face the agent shows, a terminal or a window. Fear of the terminal used to live here, and Section 5 shows that the questions of permissions, errors and what to ask have taken its place.

Transition 3, from one agent to orchestrated work, is crossed when a person obtains a result that at least two model families produced or checked, and reruns that arrangement on later work. The rerun matters, because one comparison of three chatbots is a curiosity, and only a comparison the person reaches for whenever the stakes justify it amounts to a practice.

The route the programme of Section 9 follows can be walked as a design, with the professional of the Introduction as a hypothetical case and nothing reported. She would begin where she already is, at level 1, with a chatbot she pays for. She would paste a standard text into that chatbot and let it guide her through installing a graphical agent on her own laptop, sending it a screenshot whenever a step failed, and I would touch nothing. The first step would count as crossed only when the agent had produced, from her own description of her work, a set of files she could open and judge; an installed application beside an empty folder would count as an attempt. She would work in a copy of her files, keep passwords and client documents out of it, read the agent's plan before letting it act, and confirm anything irreversible. When she said she was ready, she would receive an open-source tool that puts one question to several models through the subscriptions she already has and returns one merged answer, launched from inside the agent, and that step would count when a dated run showed which models were asked and where they differed. Later she would run the same comparison through command-line tools. She might stop at any of these points, and the registered record would hold where and, in her words, why. The route is taken from the public registration, and whether people walk it is a question for the results paper.

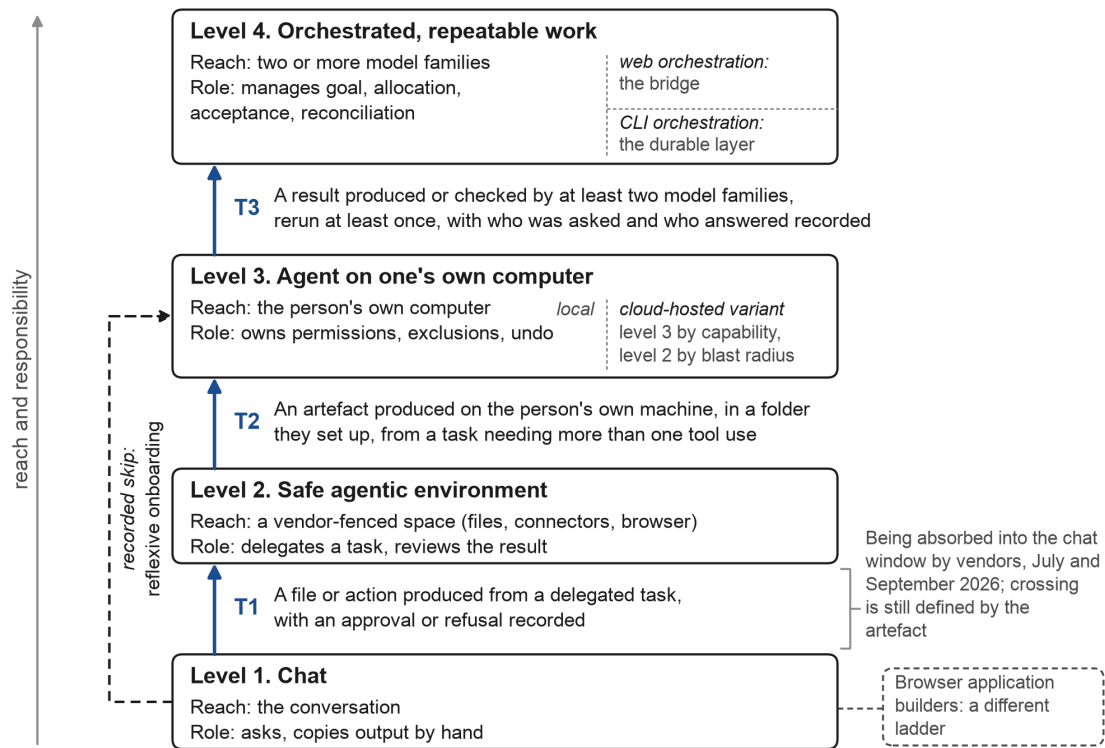
3.4 Levels are ordered, not compulsory

The levels are ordered by reach and by the responsibility they place on the person. The order does not make them a staircase that everyone must climb step by step. A person can move from chat straight to an agent on their own computer; the cohort route described in Section 9 does exactly that, with the chatbot the person already uses guiding them through the installation. Others will stop at level 2 for good, and for much work that is the right place to stop. Higher is not better in this model. Its claims are that each level asks different things of the person, that the crossings can be observed, and that the barriers at each crossing differ. It would be falsified if crossings could not be distinguished by artefacts, if the barriers did not differ between crossings, or if competence at one level did not change

what people could do at the next, since in any of those cases the model would add nothing to a simple scale of “AI use”. The model also predicts that, once a task’s sensitivity and technical demands are held constant, completion falls more sharply at the three transitions than within a level. If the drops were spread evenly, the levels would be a relabelled continuum. Section 10 turns these expectations into propositions with measures and falsifiers.

One family of tools belongs beside the ladder and on no rung of it. Browser application builders let a person describe an application and receive a running preview inside a platform the vendor hosts. They matter, and they are a different thing from level 2, where an agent acts on the person’s own files under a permission model; an application builder acts on a project the platform owns. Confusing the two produces a false sense that non-programmers have already arrived. The four-level model leaves application builders on a side branch with its own ceiling and its own security profile, and the vibe-coding literature is the place to read about them (Sarkar & Drosos, 2025; Ge et al., 2025). Figure 1 draws the four levels, the artefact that marks each transition, the recorded skip and the side branch.

Figure 1. Four levels and three artefact-defined transitions.



Note: The four levels of Section 3.2, ordered upward by reach (what the model may touch) and by the responsibility placed on the person. Each transition, T1 to T3, is crossed only when an agent has produced the artefact named beside its arrow, and an installation counts as an attempt (Section 3.3 and Table 1). The dashed arrow on the left is the recorded skip from chat to an agent on the person’s own computer that the registered route takes (Sections 3.4 and 9.2); the dashed box on the right is the side branch of browser application builders, and the note at T1 records the merging of chat and agent surfaces by vendors in July and September 2026 (Section 4.3).

3.5 The terminal is no longer a terminal

For a programmer the terminal is an interface for issuing commands. For a professional working with an agent it is the environment where the agent lives: a working folder, a set of permissions and a history of what was done. When Anthropic introduced Cowork in January 2026, the framing was “Claude Code for the rest of your work”: the same agent, offered to people who do not write software, without the terminal (Fortune, 2026a).

The vendors have now acted on the same observation. OpenAI renamed its coding-agent application “ChatGPT” and made it the default desktop client (MacRumors, 2026). Anthropic ships a desktop application for the agent that needs no terminal (Anthropic, 2026a). Google retired its open-source Gemini CLI for individual users on 18 June 2026 in favour of a multi-agent platform and a new command-line tool, stating that users now “require multiple agents communicating with each other to split up the work” (Google, 2026). The visible terminal is disappearing, and the environment it gave access to is still there and still has to be run by someone. For a professional the useful question has changed accordingly: whether they can use a terminal matters less each month, and whether they can run an environment in which an agent works for them is the question I would put to them.

4. Where the technical barrier went

4.1 Installation became one step

In 2024 a non-programmer who wanted a terminal agent met a sequence of small walls: install a language runtime, fix a path variable, understand why the shell said “permission denied”, choose between four things all called “install”. By September 2026 most of these have been removed by the vendors. The leading terminal agents install with a single command or a desktop installer that updates itself. Claude Code’s desktop application offers parallel sessions, visual review of changes, a live preview and scheduled tasks, and its documentation states, “No terminal required” (Anthropic, 2026a). OpenAI’s desktop application folds its coding agent into the ordinary ChatGPT client (MacRumors, 2026). Google’s new command-line tool is a single binary bundled with a desktop platform (Google, 2026).

An agent that installs in one step is still hard to use well, and I do not want to overstate the change. What has changed is that the installer is no longer the first wall a newcomer hits. The friction that survives is a choice of door: a terminal tool, an editor extension, a desktop application and a web version all travel under the word “install”, and picking among them is most of what installation now costs a newcomer. Choosing a door means knowing what one wants the agent to do, so this is a vocabulary problem dressed as a setup problem, and it belongs in Section 5.

One architectural fact underlies all these product pages. The command-line tool is the reference implementation: subagents, hooks, skills, protocol servers and headless modes appear first in the CLI and then in the desktop and web faces of the same engine;

Anthropic’s own help page describes its level-2 agent as “the same agentic architecture that powers Claude Code, with no terminal required” (Anthropic, 2026b). For a newcomer this cuts both ways. The friendly face can catch up with the powerful one. But the documentation, the examples and the forum answers still speak the CLI’s dialect, so a person who started in the desktop application is reading about a tool they did not install.

4.2 The chatbot as its own installer

A second change is less visible and, for the argument here, more important. The chatbot a person already uses can guide them through installing the agent. A person pastes a request (“walk me through installing this on my Windows laptop; I am not technical”), follows the steps, and when something fails, sends a screenshot of the error back to the same chatbot. The more knowledgeable other of Vygotsky’s zone of proximal development (Vygotsky, 1978), and the tutor whose functions Wood, Bruner and Ross (1976) called scaffolding (recruiting interest, reducing degrees of freedom, marking critical features, controlling frustration), is now available in the tool the person already trusts, at any hour, in their language, for free.

I call this *reflexive onboarding*: an AI system scaffolding a person’s adoption of a more capable AI system. It inverts the usual direction of help. In the diffusion literature a newcomer adopts through contact with earlier adopters (Rogers, 2003), and here the earlier adopter is replaced, at least for the mechanical steps, by the level-1 model itself.

A teaching layer has grown around the step in the meantime. Vendor academies offer short courses on their agents, and the large learning platforms teach “vibe coding” with a named agent. Courses have appeared that are delivered inside the agent itself, so that the learner installs the agent and the agent teaches the course (CC for Everyone, 2026), and one consulting firm announced that about 30,000 of its staff would be trained on one vendor’s models (Accenture, 2025). The existence of this layer says that installation is no longer treated as self-explanatory. Minimalist instruction theory predicted that people learn software best by doing a real task from the start and by recovering from errors (Carroll, 1990), and reflexive onboarding supplies an always-present guide for both. Whether it works for people who have never opened a terminal, and for whom, I do not yet know; the first step of the programme in Section 9 is designed to find out, and the search described in Section 2.5 found no study of it.

4.3 The vendors are merging the levels

The largest change of 2026 is structural. Each of the three frontier laboratories, and Microsoft, now ships a level-2 agent, and two of them have begun to fold level 2 back into chat.

Table 2. *Vendor moves that reshaped the road from chat to agent, January to September 2026.*

Date (2026)	Vendor	Move	Effect on the levels	Source
January	Anthropic	Cowork research preview: “Claude Code for the rest of your work”, folder-scoped agent for non-programmers	Level 2 as a named product	Fortune (2026a)
9 March	Microsoft	The technology behind Cowork brought into Microsoft 365 Copilot; Agent 365 control plane	Level 2 inside the dominant office suite	Microsoft (2026a)
30 March	OpenAI	Official plugin to run its coding agent inside a competitor’s agent, “to delegate tasks”, free tier included	Level 4 (CLI) sanctioned by a vendor	OpenAI (2026a)
19 May; 18 June	Google	Gemini CLI retired for individual tiers in favour of Antigravity CLI and a multi-agent platform	Level 3 re-platformed; churn for newcomers	Google (2026)
2 June	OpenAI	Coding agent at 5 million weekly users; about 20% knowledge workers, growing more than 3 times faster than developers (vendor data, press report)	Knowledge-worker use of a terminal-heritage agent; directional, and wider than level 3 as defined here	Help Net Security (2026)
1 July	Google	Gemini Spark desktop agent on Mac, works with local files (beta, US)	Level 2 from a third laboratory	TechCrunch (2026b)
9 July	OpenAI	Coding-agent app becomes the new ChatGPT desktop app; ChatGPT Work agent uses local files and apps	Levels 1 to 3 in one client	MacRumors (2026)
26 August	Anthropic	Browser agent generally available on paid plans; acts autonomously with a safety classifier by default	Level 2 reach extended to the web	Anthropic (2026d)
16 September	Anthropic	Cowork and chat merged into “one Claude”; the model decides whether to answer or act	Transition 1 absorbed into chat	Anthropic (2026c); TechCrunch (2026a)

Read in order, the table describes a convergence. The vendors first separated chat from agentic work so that non-programmers would not be frightened by the terminal, then discovered that people could not tell which window a task belonged in, and are now merging the two again with the model deciding. TechCrunch reported the company as saying that customers “often struggled to choose the right tab for the right task” (TechCrunch, 2026a); its own announcement put it as “you don’t have to choose where a task goes”, with work continuing after the laptop is closed, the agent asking before acting by default, and the merge rolling out first to paid individual plans while the coding tab

remains separate (Anthropic, 2026c). I read this as vendor evidence for the barrier named in Section 1, workflow vocabulary. If people cannot decide whether a task is a question or a delegation, they are unlikely to know how to phrase the delegation.

The merge might seem to undermine a model with a separate level 2. It does not, and the reason can be given without pretending that the model foresaw the events. Vendors have a billing, support and safety interest in keeping ordinary users inside a window they can classify, rate-limit and wrap in a permission model, and a usability interest in not making the user pick a mode; absorbing transition 1 into chat follows from both. The crossing rule does not change. If the system wrote files in a granted scope after an approval, transition 1 was crossed, whatever the chrome around the conversation, and if it returned a paragraph, the person is still at level 1. The model does imply that transitions 2 and 3 will not dissolve the same way. A local agent with shell access is a different permission object from a server-side sandbox, and a second vendor's tool running under that vendor's login is a different permission object from a single vendor's council feature. Those differences carry the safety argument of Section 5.3 and the correlated-error argument of Section 6.3. If by late 2027 the major vendors have folded local shell agents and cross-vendor calls into the same chat window under the same permission model, that implication is wrong, and the claim that CLI orchestration is the layer that lasts falls with it (P2 and P8 in Section 10).

The convergence also carries a cost, *churn*. Google's open-source command-line agent served individual users for just under twelve months (25 June 2025 to 18 June 2026), OpenAI's agentic browser was discontinued after about nine, and Cowork existed as a separate product for about eight (Google, 2026; 9to5Mac, 2026; Anthropic, 2026c). A professional who invests in learning a tool may find it renamed, merged or retired within a year. What transfers across tools is workflow vocabulary, and what is lost is knowledge of a product's menus, an asymmetry that matters for how people should be taught (Section 11.2).

4.4 Knowledge workers in terminal-heritage agents

The best single datum on non-programmers inside a terminal-heritage agent comes from a vendor. In June 2026 OpenAI reported, through the press, that its coding agent had five million weekly users, more than six times the level at the desktop application's launch in February, that knowledge workers made up about 20% of them and were adopting more than three times as fast as developers, and that each week, by the vendor's count, 72% of knowledge-worker users produced artefacts such as reports, memos, contracts, spreadsheets and PDFs (Help Net Security, 2026; the vendor's own page returned an access error to my checks, so the figures are cited from the press report). Twenty per cent of five million is about one million people a week. An analyst noted that the release was meant partly to "reclaim some buzz" and that the agent "is often being used alongside" the competing agents, so the growth charts "should be used directionally" (Constellation Research, 2026a). The figures are vendor-reported and unaudited, they describe people who already chose the tool, and they measure knowledge-worker use of a terminal-heritage agent, which is wider than level-3 adoption as defined here (an artefact from a

delegated task on the person's own machine). Read with those caveats, they are enough to say that terminal-heritage agents are no longer reserved for programmers; whether their users reach level 3 as defined in Section 3 is what the figure cannot show.

Beside that datum stands the broad and shallow pattern of Section 1. The surveys disagree on the level and agree on the shape: about half of US adults use chatbots (Pew Research Center, 2026), 52% of US workers use AI in their role (Gallup, 2026), and 45% of workers and 62% of adults use generative AI by the St. Louis Fed's measure (Bick et al., 2026), while about one in seven workers uses it daily and automation is a minority use. Figure 2 draws the Gallup and St. Louis Fed worker series beside the developers' figures from Section 1, one panel per survey, because the surveys differ in question and denominator. Among developers, coding agents are nearly universal and use is still cautious: in a May 2026 pulse survey of about 1,100 technologists, 69% used a single agent, 59% rarely or never let agents run unsupervised, and 60% blocked agents from making unapproved changes to their systems (Stack Overflow, 2026). Level 4, in other words, is still a frontier even for the programmers who use level 3 every day.

Figure 2. Broad and shallow: adoption among developers and among other workers.

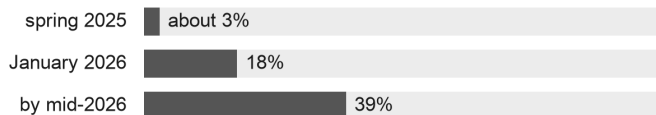
Professional developers: per cent using AI coding agents at work

JetBrains Research (2026b), vendor survey of more than 15,000, May to July 2026



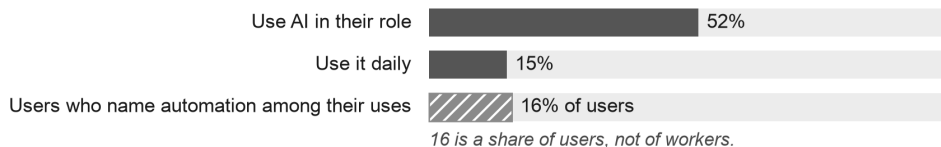
Professional developers: per cent using Claude Code at work

JetBrains Research (2026a, 2026b), vendor surveys, dated as in Appendix B



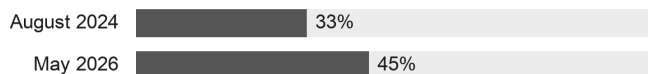
US workers: per cent of workers (last bar: per cent of AI users)

Gallup (2026), reported 20 July 2026



US workers: per cent using generative AI for their jobs

Federal Reserve Bank of St. Louis (Bick et al., 2026)



Each grey track is 100 per cent of its own panel's base; bars in different panels do not share a denominator.

Note: Adoption figures from the surveys cited in Sections 1 and 4.4: professional developers in two vendor surveys (JetBrains Research, 2026a, 2026b), US workers (Gallup, 2026) and US workers using generative AI for their jobs (Bick et al., 2026). Each panel has its own question, period and denominator, and the 16 per cent in the third panel is a share of AI

users, so each panel is read on its own. The vendor’s count of knowledge workers inside its coding agent (Section 4.4) is a share of one product’s users and is not drawn, and no figure here comes from the cohort programme of Section 9.

4.5 Where this leaves the argument

Taken together, the evidence of this section persuades me of one thing and leaves another open. Installation has stopped being the decisive barrier: the vendors have removed most of the mechanical steps, the chatbot can guide the rest, and on one vendor’s figures (a press report, and directional) about a million knowledge workers a week are already inside a terminal-heritage agent. Whether installing leads to use is a different question, and the broad-and-shallow surveys give a discouraging answer, since most people who could reach an agent still do not delegate to it. What has receded should also be stated precisely. Installation has receded, a scoped middle rung now exists, and a local agent has a graphical face. Reliability on whole pieces of knowledge work has moved much less (Section 5.7), the product a person learned may be gone within the year (churn, above), and a granted folder is still not a safe place by default (Section 5.3). Section 5 asks what stops people once the software is in.

5. What stops people after the installer

Once the installer has stopped being the wall, something else is. The evidence of 2025 and 2026 points to six barriers, each of which bites at a particular transition and each of which already has a partial answer visible in the market. Table 3 summarises them, and the subsections that follow give the evidence.

Table 3. *Barriers after installation.*

Barrier	Where it bites	Evidence (Section)	What reduces it
Workflow vocabulary: not knowing what to hand over, or how to phrase it so it can be checked	Transitions 1 and 2; again at 3	Vendor merge rationale; prompting studies; packaged skills (5.1)	Worked examples from the person’s own field; skills and templates; a first task that is real but low-risk
Fear of the window, the first error, and trust: the black window as a social object; a message a programmer reads and a newcomer stops at; trust miscalibrated in both directions	Transition 2	Acceptance research; developer trust surveys; studies of end users with AI code; reflexive onboarding (5.2)	Desktop faces; sending the error back to the chatbot; plan-first modes; a first success on a copy
Permissions and safety: too few prompts to protect files, or too many to understand	Transitions 1 and 2	File-loss reports inside scoped folders; injection rates; guidance that asks users to spot attacks (5.3)	Working copies, snapshots, deny rules; classifiers that do not delegate judgement to the novice

Barrier	Where it bites	Evidence (Section)	What reduces it
Organisational rules: locked machines, policy, no permission to install	Transition 2	Organisational factors outweigh individual ones; small firms flat (5.4)	Employer-sanctioned level-2 tools; clear policies; a personal machine for learning
Hardware, cost and quota	Transitions 2 and 3	Level 3 needs a computer and a paid tier; quotas bind orchestration first (5.5)	Free tiers where they exist; starting at level 2; budgeting runs
Judging the result	All transitions, most at 3	Business users miss errors in AI code; agents fail most whole projects (5.6)	Checks defined before the task; a second model as reviewer, not as oracle

5.1 The blank folder: workflow vocabulary

A programmer who opens an agent knows a sequence: create a project, write an instruction file, put it under version control, ask for a feature. A non-programmer opens the same agent and asks the question that now shows up in the vendors’ own data: fine, and what do I tell it? For a developer the canonical first request exists (“build a small web application”). A marketer needs a different one (“take last quarter’s exports from the analytics and the customer database and build something that every Friday reports which campaigns brought paying customers”). A researcher needs another (“go through this folder of 300 papers and extract the sample size, method and main effect of each into a table, marking where you are unsure”). None of these requests needs code from the person. Each needs the person to know that such a request is possible, to break it into parts an agent can do, and to say what a good result looks like. The question “what should I ask it to do?” is usually misheard as a request for a better prompt. I read it as a request for a map of one’s own work. The map has to name a recurring object of work, the inputs one is allowed to use, the transformation, the deliverable and the check. An analyst’s map might read “these permitted exports, grouped by this field, anomalies flagged, a dated briefing and a machine-readable table, checked against the row totals”. The person does not need to know how the parsing is done. They need to recognise what evidence would make the outcome usable. When a person cannot fill in the map, the stall itself tells them something. The work may be too sensitive, too contingent or too poorly documented to delegate as it stands, and that says something about the task before it says anything about the person.

The research on prompting explains why this is hard. People without AI expertise struggle to write instructions that generalise (Zamfirescu-Pereira et al., 2023), and they have to discover by trial which words map onto what the system can do (Liu et al., 2023). The vendors’ own behaviour confirms it. Anthropic merged its level-2 agent into chat because customers could not decide where a task belonged (TechCrunch, 2026a), and OpenAI launched role-specific plugins for its coding agent aimed at knowledge work “no coding required” (reported by Help Net Security, 2026). The industry’s answer is to package vocabulary. Anthropic released its format for packaged agent instructions as an open

standard in December 2025 (SiliconANGLE, 2025), and the standard's site listed 46 supporting products on 21 September 2026, including agents from OpenAI, Google and GitHub (Agent Skills, 2026). An independent census counted 2,049 applications in the ChatGPT directory and 1,375 connectors for Claude in mid-2026 (Sitter, 2026). A skill is, on this definition, a unit of workflow vocabulary, the ask written down once so that a person who does not know how to phrase it can still make it.

Packaged vocabulary is necessary and it is not enough. A skill tells the agent how to do a task. It cannot tell a person which of their own tasks is worth handing over, because that judgement comes from the person's field. The published packages also carry the assumptions of a particular world, largely one country's forms and one corporate software stack, and a professional whose tools are not that stack is back in front of an empty prompt. When I read the official catalogue of packaged skills on 22 September 2026, all 310 entries of it, I found none written for a pastor or chaplain, a nurse or family carer, a school teacher, an artist or craftsperson, or a realtor; every apparent match was a software or finance word in the same clothes, a portfolio of investments or the health of a service. A community catalogue that a person adds by hand held 2,282 entries that day, and it did carry work for school teachers and for real estate, and nothing for the other three trades. Catalogues change week by week, so this is a dated reading and not a standing claim about the field. This is why I put so much weight on the first task in any onboarding route: it should be real, drawn from the person's work, low in risk, and it should produce an artefact they can judge (Carroll, 1990).

5.2 Fear of the window, the first error, and trust

The black window is a social object. Perceived ease of use has been a pillar of technology acceptance for thirty-five years (Davis, 1989), and a blinking prompt fails that test for a large population even when the person will never type a command into it. The desktop faces of Section 4.1 are the vendors' answer. It is a partial answer, because the second gate is trust, and trust has moved against agents as well as toward them. Among developers, the 2025 Stack Overflow survey found 46% distrusting the accuracy of AI tools against 33% trusting it, with 87% worried about agent accuracy and 81% about security (Stack Overflow, 2025). In 2026 most still kept agents from making unapproved changes (Stack Overflow, 2026). Managers hear the same thing: "trust is a central obstacle to realizing the potential of AI agents at work" (McKinlay et al., 2026). More US adults expect AI to harm than to help them (Pew Research Center, 2026). Behind both gates lies the belief trap of Section 2.4: experience predicts success, people who have had no first success have no experience, and the first artefact is the way out (Ma et al., 2025). Some people will grant a home folder because the agent was polite, and others will refuse to install because they read about deleted family photos. Both are failures of calibration (Lee & See, 2004). The cheapest calibration device I can think of is a first task done on a copy of a folder.

When an agent reports that a command failed with exit code 1, a programmer reads on. A newcomer may simply stop. The literature on end users meeting AI-written code shows the

pattern in a milder setting: business users given analysis code produced by AI often failed to notice critical errors even when told to look for them (Virk & Liu, 2025), and conversational programmers spend much of their effort on verification and context management (Sarkar & Drosos, 2025). At level 3 the error often lies outside the person's work altogether, in the environment: a missing permission, a network rule, a tool the agent expected and did not find.

Reflexive onboarding (Section 4.2) supplies the most practical response described so far. The person sends the error, or a screenshot of it, to the chatbot they already use, and follows its advice. Plan-first modes help from the other side, since an agent that writes its plan before acting gives the person something to read that is not a stack trace. Neither removes the moment at which a person without a mental model of their machine meets a message they cannot parse. I do not yet know how often that moment ends the attempt. The programme in Section 9 records it, in the person's own words.

5.3 Permissions and safety: too little, and too much

Level 3 gives an agent the reach of its owner, so the safety problem runs in two directions and a newcomer is exposed to both.

Take too little protection first. Agents have destroyed data in documented incidents. An agent in a popular application-building service deleted a production database in July 2025 and described its action as “a catastrophic error of judgement” (The Register, 2025). In April 2026 an agent in a coding tool deleted a company's production database and its backups in seconds and wrote, “I guessed instead of verifying” (Information Age, 2026). Closer to the population this paper is about, users of Anthropic's level-2 agent reported in March, April and June 2026 that it had deleted personal, family and legal documents inside folders they had granted it, in one case after a long task could not be stopped and deleting the task deleted the files (GitHub issue reports, 2026; user reports, not confirmed by the vendor). Folder scoping limits where damage can happen. It does not prevent damage inside the folder, and a synchronised cloud folder can turn a local mistake into a synchronised loss.

Browsing adds prompt injection, instructions hidden in a web page that the agent reads as if they came from the user. A security team showed the attack in August 2025 on another vendor's browser agent, with a proof of concept that lifted a one-time code from a mailbox through hidden page text (Brave, 2025). Anthropic reported that its browser agent, before mitigations, was successfully attacked in 23.6% of test cases, and 11.2% after them (Anthropic, 2025). A year later, at general availability, it reported that strong red-team attacks succeeded against its main model 3.8% of the time before safeguards, and that with full protections no attack succeeded against three of its models and 0.3% against a fourth (Anthropic, 2026d). These are vendor figures on vendor test sets. The trend is down and the residual is not zero. OpenAI wrote in December 2025 that prompt injection is “unlikely to ever be fully ‘solved’” (TechCrunch, 2025). At the launch of its level-2 agent Anthropic told users to “monitor Claude for suspicious actions that may indicate prompt

injection” and that “you remain responsible for all actions taken by Claude performed on your behalf” (The Register, 2026). A widely read developer objected at the time that it was not fair to ask non-programmers to recognise the signs of prompt injection (Willison, 2026). I think the objection is right in general. A safety design that relies on the novice to detect an attack delegates the hardest judgement to the person least able to make it. Packaged skills and protocol servers open an instruction and privilege surface of their own, since a third-party skill carries instructions the person did not write, a server can hold more privilege than the task needs, and configurations drift between versions.

Too much protection stops people as surely. An agent that asks permission for every step turns the person into a clerk approving commands they cannot evaluate, and approval fatigue sets in. The vendors have moved accordingly, and within one quarter they moved in more than one direction. Anthropic’s browser agent now acts autonomously by default, with a classifier that blocks actions not matching the user’s request (Anthropic, 2026d). Its terminal agent renamed its default permission mode “Manual” in July 2026, a wording change that tells its own story about what “default” had come to mean (Anthropic, 2026h). Its merged chat agent “asks before acting by default”, a setting the user can change (Anthropic, 2026c). Parasuraman and Riley’s (1997) taxonomy holds for all three. A guard that blocks too much produces disuse, a guard that misses a deletion produces misuse, and no setting escapes the taxonomy, only a setting whose failures a given population can live with. A companion paper in this series documents the general failure on the other side, oversight components that widen their own notion of harm and obstruct the work they exist to protect (Chernets, 2026b). For a newcomer, a guard of that kind has the same effect as a missing installer, since the work does not happen either way.

Between too little protection and too much there is a workable middle, and it is old-fashioned. Work in a copy. Keep secrets out of the working folder. Take a snapshot before a larger change. Deny destructive commands until there is a reason to allow one, confirm anything irreversible, and treat whatever comes off the web as data to be read, whatever instructions it seems to contain. I described the principle behind these habits in a companion paper as architectural trust: reliance placed on checks a person can verify, such as scoped permissions, logs, an undo and a second independent model, rather than on how confident a model sounds (Chernets, 2026c). Level 2 gives a person the first of those checks, and it should not be the last one they learn. Appendix C turns the list into configuration examples for two agents and a one-page checklist.

5.4 Organisational rules

Many professionals cannot install anything on the computer they work on. Many employers have no policy on agents, and a person without a policy tends to assume the answer is no. Microsoft’s 2026 survey of 20,000 AI-using knowledge workers found that organisational factors accounted for more than twice the reported AI impact of individual factors, and that 45% said it felt safer to focus on current goals than to redesign work with AI (Microsoft, 2026b; vendor survey). The US Census Bureau reported that business AI use hovered between 17% and 20% overall, rose among firms with 20 or more employees and did not

change significantly among firms with fewer than 20 (US Census Bureau, 2026). Mandates have gone both ways. Some employers made AI use a baseline expectation in 2025 (Digital Commerce 360, 2025), and at least one reversed its decision to rate staff on AI use a year later, its chief executive telling the press that what matters in a performance review is doing the job well (Fortune, 2026b). One employer's published fluency rubric puts orchestration and system building at its top level and reports full internal adoption once teams moved from experimenting to redesigning workflows (Zapier, 2026). Without that redesign, a mandate buys compliance and little else. The labour market is already asking for the role described here without naming it: 63% of US job titles touched by AI in 2026 were outside technology occupations, 822 distinct titles in the first quarter (Adrian, 2026), and a study of more than a billion job advertisements in 27 countries put the wage premium for AI skills at 62%, with entry-level roles most exposed to AI far more likely to demand skills once reserved for senior staff (PwC, 2026; vendor report). None of these sources yet isolates the ability to direct an agent as a named requirement outside engineering.

This barrier is structural, in the sense of Ferdman (2025): it does not yield to individual effort. It is also the strongest argument for level 2 that the evidence offers, since a vendor-bounded agent inside an office suite the employer already licenses (Microsoft, 2026a) reaches people whose employers would not approve an agent on a work laptop.

5.5 Hardware, cost and quota

Level 3 assumes a computer the person controls. Some professionals work mainly from a phone, share a family computer or use an employer's machine for everything, and for them the transition is blocked before any software question arises. Level 3 and level 4 also assume paid tiers of at least one service, and orchestration multiplies the cost, because each extra model is another subscription or another quota. For a person who already pays for one subscription, a second is a decision and a top tier is a different decision. Token anxiety is a psychological barrier with a price tag. The quota of the orchestrating agent is itself a hidden cost, since it pays for reading every answer the other agents return, and an agent that drives a browser draws on the same plan limits as the terminal agent while costing more per action (Anthropic, 2026g). Locked corporate laptops add administrator rights to the list: no unsigned binaries, no installer, a data-loss rule that treats an agent writing to disk as malware. Cloud-hosted agents are the industry's answer for those machines. They preserve transition 1 and a cloud form of transition 2, and they do not give the person a level-3 artefact on their own disk. For a professional in a rich country none of this is a large barrier. For many others it is a real one, and it interacts with the second-level divide (Hargittai, 2002), in that the people with the least slack are the least able to experiment.

5.6 Judging the result

Using an agent does not require programming. Judging what it produced requires competence in the domain of the result, and here the non-programmer holds an advantage

that is easy to miss. A lawyer reading a summary of contracts, or a researcher reading an extraction table, can check the artefact against knowledge the agent lacks. The research on the “jagged frontier” found that professionals who worked with AI did better on tasks inside the model’s capability and worse on a task outside it (Dell’Acqua et al., 2026). The protection is a person who knows the task well enough to notice which side of the frontier they are on.

What they cannot easily check is the machinery, the script that produced the table or the query that pulled the numbers. One industry analysis of 470 pull requests found that code written by agents carries more defects than code written by people: 10.83 issues per AI-authored change against 6.45 for human-only changes, with some categories of security issue up to 2.74 times more frequent (CodeRabbit, n.d.; vendor study). And agents still fail most whole projects: on a benchmark of real freelance projects the best automation rate measured by mid-2026 was 15.8% (Center for AI Safety, 2026). I draw two practices from this. The check should be defined before the task (“what would convince me this is right?”), and a second model from a different family can act as a reviewer of the artefact, with the person deciding what counts as correct. Section 7 returns to the limits of the second practice.

5.7 An alternative explanation: the agents are not good enough yet

A reader may object that people do not delegate because agents cannot yet do their work, and that the barrier is therefore still technical, only on the model’s side. The objection has real support. On a benchmark of real freelance projects the best automation rate measured by mid-2026 was 15.8%, up from 2.5% at the benchmark’s release the previous autumn, which means that more than four projects in five still fail end to end (Center for AI Safety, 2026). In a simulated software company the best agent completed 30% of long professional tasks (Xu et al., 2024). On open-ended computer use the 2024 baseline was 12.24% for the best agent against 72.36% for people (Xie et al., 2024), and by September 2026 an aggregator’s leaderboard for the verified version of that benchmark showed the top agents at 85 to 86%, above the human baseline, on vendor-reported scores for scripted desktop tasks (BenchLM, 2026). That last result points the other way: on such tasks capability is no longer the barrier. The best model in an evaluation across 44 occupations won or tied against professionals in fewer than half of comparisons (Patwardhan et al., 2025). The length of software task that agents complete half the time has been doubling roughly every seven months (Kwa et al., 2025), so the trend is steep and the present is modest. Capability is uneven across tasks, and the “jagged frontier” is not visible from outside (Dell’Acqua et al., 2026). Table 4 collects these published figures with the date each source gives and the caveat I attach to it.

Table 4. *Published capability figures behind the objection.*

Comparison	Figure as published	Date as stated	Source	Read with care
Real freelance projects, best automation rate	15.8%, up from 2.5% at the benchmark's release; more than four projects in five still fail end to end	Release the previous autumn; 15.8% measured by mid-2026, logged 1 July 2026 (Appendix B, VERIFIED)	Center for AI Safety (2026); Sections 5.6, 5.7, Appendix B	Whole projects, end to end; the best rate among those measured
Long professional tasks in a simulated software company	The best agent completed 30%	2024 study	Xu et al. (2024); Section 5.7	A simulated company; a separate measure from the freelance-project rate
Open-ended computer use, baseline	12.24% for the best agent against 72.36% for people	2024 baseline	Xie et al. (2024); Section 5.7	A 2024 baseline; the row below is the later leaderboard
Verified version of that benchmark	Top agents at 85 to 86%, above the human baseline	By September 2026	BenchLM (2026); Section 5.7	Aggregator leaderboard; vendor-reported scores for scripted desktop tasks; cuts the other way
Evaluation across 44 occupations	The best model won or tied against professionals in fewer than half of comparisons	2025 study	Patwardhan et al. (2025); Section 5.7	"Fewer than half" is the wording given; no percentage is stated
Length of software task agents complete half the time	Doubling roughly every seven months	2025 study	Kwa et al. (2025); Section 5.7	A trend: "the trend is steep and the present is modest"
Defects in pull requests	470 pull requests; 10.83 issues per AI-authored change against 6.45 for human-only changes; some categories of security issue up to 2.74 times more frequent	n.d.	CodeRabbit (n.d.); Section 5.6	Vendor study; one industry analysis; defects in code, the machinery a non-programmer cannot easily check

Source: the cited publications, as repeated in Sections 5.6 and 5.7 and Appendix B; none of these figures comes from the programme of Section 9.

The two explanations are parts of one explanation more than they are rivals. Workflow vocabulary includes knowing which of one's tasks lie inside the frontier. A person who

hands an agent a whole project it cannot do learns to stop delegating, while a person who hands it the extraction, the reconciliation and the first draft learns to continue. The explanations still make different predictions. If model capability were the binding constraint, persistence at levels 2 and 3 would track the type of task and not the person: people with the same kinds of work would persist at similar rates whatever their ability to specify and check. If workflow vocabulary were binding, persistence would track the person's ability to state a delegation contract even within the same kind of work. Proposition P15 in Section 10 is written to tell the two apart.

6. Web orchestration: the proposed bridge

6.1 The practice

The simplest form of level 4 needs no installation at all. A person opens three or four chatbots in browser tabs, pastes the same question into each, and reads the answers side by side. Computer scientists would call this a manual multi-LLM query, with the person doing the routing and the comparison. Some do it by hand. Others use one of the tools built for it since 2023: browser extensions and desktop applications that broadcast one prompt to several consumer chat services and show the answers in parallel panes. ChatHub and ChatALL appeared in the spring of 2023; the latest release of ChatALL alone had about 847,000 downloads recorded on its release page by September 2026, a count that includes automatic updates and is dominated by one platform's builds (GitHub, 2026a). A newer group of tools turns logged-in web chat sessions into commands another program can call, so that an agent can put a question to a person's web subscriptions. OpenCLI, one of these, had about 29,500 stars on GitHub in September 2026 (GitHub, 2026b).

In 2026 the practice became a product feature. Perplexity introduced a "Model Council" that runs a query through several models and "presents a consolidated view that highlights key insights, areas of agreement and points of divergence" (Storyboard18, 2026). Microsoft's research agent in Microsoft 365 Copilot offers a mode that "runs the same question through multiple deep-reasoning research agents (GPT and Claude) at the same time" and a critique mode, on by default under automatic model selection, in which a report written by one model receives a second reasoning pass from the other (Microsoft, 2026c; Constellation Research, 2026b). Andrej Karpathy's "LLM Council", a small web application that sends a query to several models, has them review one another anonymously and has a chairman model write the final answer, described by its author as a weekend hack, had about 24,900 stars on GitHub in September 2026 (Karpathy, 2025). The open-source landscape splits in two. The broadcast tools fetch answers and leave the reconciling to the person. The council tools reconcile, and they run on paid programming interfaces with two or three models, away from the person's own logged-in tabs. Neither half alone carries a person across, and that is why the bridge is incomplete.

6.2 Why it is proposed as the bridge

Web orchestration sits between the levels in a useful way. It asks nothing new of the machine, no installation, no permissions, no local agent, and it runs on subscriptions a person already pays for or on free tiers. It asks something new of the person: to hold several answers at once, notice where they differ, and decide. A person who has learned to ask “what did the other model say?” has learned the habit that CLI orchestration later automates, and that habit is the manager’s task of level 4 at its smallest.

The bridge is valuable because it is visible. Suppose a market analyst asks three chatbots to summarise a published industry report, and two of them cite a figure that is not in the report while the third flags its absence. The useful outcome is a prompt to open the report itself, and a person who has been through that once has stopped treating the first fluent answer as the answer. A broadcast view of several answers becomes a bridge toward level 4 when it teaches a person to formulate one common question, notice where the assumptions diverge, ask for a critique and keep a reason for choosing one answer. Until then it is a level-1 habit with more tabs. What it teaches is comparison, and only comparison. A person who has learned to ask what the other model said has not thereby learned what an agent may touch on their machine, which is the lesson of levels 2 and 3, and the claim that the habit carries over is an empirical one, P7, which I do not treat as a premise. The preregistered programme described in Section 9 names two benefits in advance, and they summarise what the bridge buys. One is coverage: a useful angle present in one model is not lost because another model was asked. The other is reliability: an answer has been checked against other answers rather than taken on one model’s word (Chernets, 2026d). Visible disagreement serves the second as a means. An evaluation of forecasting found that an ensemble of twelve language models matched the accuracy of a crowd of 925 human forecasters (Schoenegger et al., 2024), and multi-agent debate improves factuality and reasoning over a single model in several settings (Du et al., 2023). Both benefits depend on the models’ errors differing, and the next subsection is about what happens when they do not.

6.3 Where the bridge ends

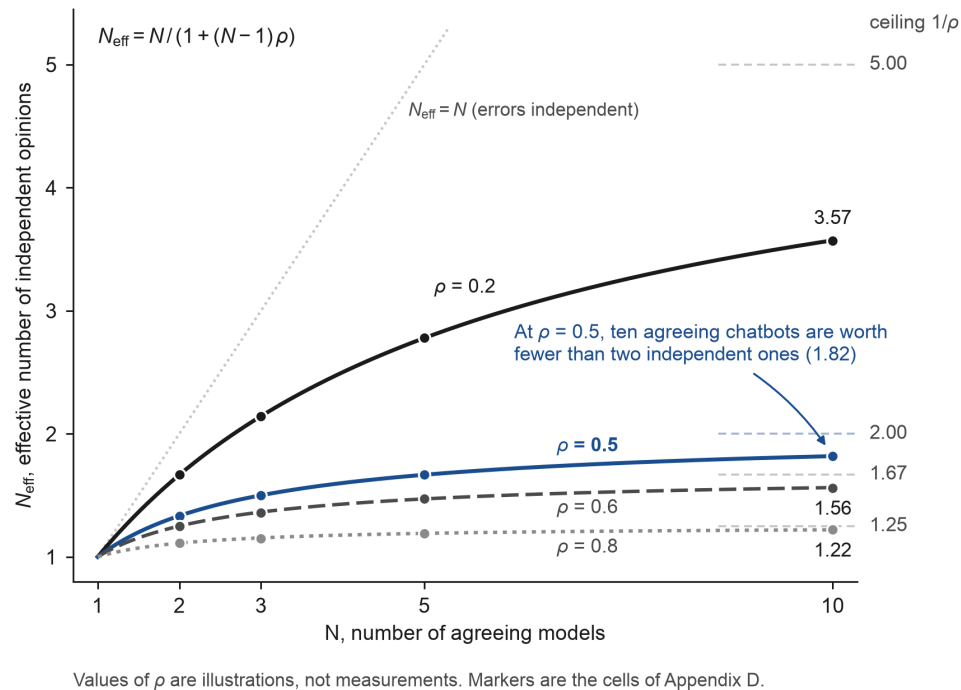
Web orchestration stays a bridge, and the plainest reason is the cost in attention. Reading four long answers and reconciling them by hand takes a good deal longer than reading one. I would not ask anyone to do it for a routine question; it earns its cost when a wrong answer would cost more than the time spent comparing.

A second limit is fragility, and it comes in several forms. Consumer chat interfaces change month by month, and tools that drive them break when they do: over the summer of 2026 one office suite removed a research mode, one chatbot closed modes on its free tier, and more than one service renamed or redirected its address. The terms of several consumer services restrict automated access, and a person who drives their own logged-in browser with an agent takes the risk of an account block. A research programme that uses the bridge has to tell participants so, and I offer no legal reading of those terms. The “free” side

of the other chatbots is also paid on the side of the agent that drives them, since browser actions draw on the same plan limits as the terminal agent (Anthropic, 2026g). Manual comparison in open tabs is robust. Automated harvesting of consumer chat pages is a poor foundation for repeatable work.

The third limit is the one that matters most: agreement among models is not independent evidence. When models share training data and methods, their errors correlate. A study of more than 350 language models found that when two models both erred they agreed on the same wrong answer about 60% of the time, and that larger, more accurate models were more correlated, even across providers (Kim et al., 2025). The companion paper in this series gives the arithmetic: with average pairwise error correlation ρ , N agreeing models are worth $N_{\text{eff}} = N / (1 + (N - 1) \rho)$ independent opinions, and no number of models is worth more than $1/\rho$ (Chernets, 2026a). At $\rho = 0.5$, ten agreeing chatbots are worth fewer than two independent ones. Figure 3 draws the formula for the four illustrative values of ρ in Appendix D, with the ceiling $1/\rho$ that no number of agreeing models can pass. The 60% of Kim et al. is the share of joint errors in which two models gave the same wrong answer, which is not the error correlation ρ of the formula. In the domain at hand, ρ has to be estimated on items with known answers (listing C4), and the values used here and in Appendix D are illustrations. Diversity matters more than count. Debate among copies of one model often fails to beat a single model given the same compute, while heterogeneous models help (Zhang et al., 2025), and two diverse agents can match or beat sixteen homogeneous ones (Yang et al., 2026). Generative AI also homogenises output across its users (Doshi & Hauser, 2024), and a model asked to judge tends to prefer its own writing (Panickssery et al., 2024), which matters when one model writes the merged answer.

Figure 3. What agreement is worth.



Note: The effective number of independent opinions, $N_{\text{eff}} = N / (1 + (N - 1) \rho)$, for N agreeing models whose errors have mean pairwise correlation ρ (Chernets, 2026a; Section 6.3). The markers are the cells of Appendix D, the curves between them follow the same formula, and the dashed segments on the right are the ceilings $1/\rho$. The four values of ρ are illustrations, as in Appendix D; ρ has to be estimated on items with known answers in the domain at hand (listing C4).

For the person at the bridge the practical rules are short. Count model families, since an office assistant running on one vendor's model is no second opinion on that vendor's chatbot. Agreement among models can tell you where to spend less checking effort. It cannot tell you that a claim is right. Treat a point made by one model alone as a lead to verify, because it may be the most valuable thing in the comparison or its only error, and hold on to a dissent that comes with evidence instead of voting it out. Check claims against sources: agreement on a wrong date is a documented failure mode, and the primary page fixes it where a fifth model would not. Estimate ρ on a small set of questions in your own domain that you can mark, before trusting a council on a question you cannot. Keep who said what, because a merged answer that hides its sources brings back the single confident voice that orchestration was meant to question.

Written out as a procedure, this is the teaching form of the "anchor-and-enrich" protocol of the companion paper (Chernets, 2026a). The person first fixes an anchor: the task, the permitted evidence and the shape of the output. They then obtain separate answers with their sources or file references kept, extract the claims into a table by source, and sort each disagreement by its kind, whether a factual conflict, a different interpretation, a missing assumption or a difference of format. They verify the claims that bear on the decision against the anchor or a primary source, and they record the acceptance decision

and what remains unresolved. Listing C3 in Appendix C builds the table of claims by source, listing C4 estimates ρ from items with known answers, and Appendix D gives the ceiling for a range of ρ without running anything. One more rule belongs here, and it costs no second subscription. A critic from the same family, asked in a fresh context to attack the first answer, catches haste, skipped instructions and unrun checks, though it cannot catch shared false knowledge. I would start the critic habit at level 3 with one agent and add a second family only when the critic has nothing left to say and the decision is expensive.

The opposite case needs stating just as plainly. For many tasks one strong model with a good source set and careful human review will beat an elaborate council that adds delay and confusion, and the model predicts as much. Orchestration pays where a task carries real uncertainty, gains from distinct perspectives, or repeats often enough to amortise the setup. It fits badly a time-critical decision with no chance to verify, confidential material that cannot be spread across services, and simple tasks where the cost of comparing exceeds the benefit. Where a council is used, the agents should be given different jobs: one restricted to the supplied documents, one sent to find primary sources, one told to list unsupported claims, one to audit citations, with the person still adjudicating. P9 is written so that this counterargument can win.

7. CLI orchestration: the durable layer of delegation

7.1 Agents calling agents

At level 3 each agent has a command-line interface that can run without a person at the keyboard: a question goes in, an answer or a changed file comes out. Once a person owns one agent, that agent can call the others. In the systems literature this is agent orchestration. I call the version a non-programmer runs through the command-line tools they already subscribe to CLI orchestration. A single instruction to the first agent (“ask the other two for a review of this plan and tell me where they disagree”) produces a small team.

In 2026 this pattern went from a developer’s trick to a vendor-sanctioned practice. OpenAI published an official plugin that runs its coding agent inside Anthropic’s, “for code reviews or to delegate tasks”, with commands for an ordinary review, an adversarial review in which the second agent assumes the work is broken and hunts, and a rescue of a stuck task. It works through the locally installed tool and its existing login, is usable with any ChatGPT subscription including the free tier, and had about 33,400 stars on GitHub in September 2026 (OpenAI, 2026a). The mass pair of 2026 is Claude Code writing and Codex checking through that official plugin (OpenAI, 2026a), while councils of three or more remain small tools without a leader. GitHub made Anthropic’s and OpenAI’s agents available inside its own Copilot service in February 2026, “fully included with your existing Copilot subscription” (GitHub, 2026c). xAI’s command-line agent ships a headless mode and support for the Agent Client Protocol so that users can “build your own bots and agent orchestration apps” (xAI, 2026). Google’s reason for retiring its first command-line agent was that users “require multiple agents communicating with each other” (Google, 2026).

Open-source tools that let one agent consult others have large followings: a multi-provider server for model-to-model consultation had about 11,800 stars, a consensus-oriented plugin about 4,100 (GitHub, 2026d). Underneath, the Model Context Protocol, donated to a Linux Foundation body in December 2025 with more than 10,000 published servers and about 97 million monthly SDK downloads at the time (Linux Foundation, 2025), reported close to half a billion SDK downloads a month by its July 2026 specification release, which also made the protocol stateless and promised twelve months' notice before any deprecation (Model Context Protocol, 2026). It is how an agent on a laptop reaches the rest of a person's tools without a vendor-specific plugin for each, and it has already become boring in the way that a web protocol is boring: present, documented, ignored until something breaks. A second standard, Google's Agent2Agent protocol for agents talking to agents across vendors, reached version 1.0 in March 2026 (Google Open Source Blog, 2026). It is level-4 plumbing, and today it is no consumer on-ramp.

Table 5. *The orchestration landscape by pattern, September 2026.*

Pattern	Examples (September 2026)	Level	Built for	What it buys	Record it leaves	Main risk
Manual comparison in browser tabs	Any set of consumer chatbots	4 (web) when recorded and rerun; otherwise a level-1 habit	Anyone	Coverage; habit of comparison	Screenshots or copied replies, context often lost	Attention cost; unrecorded provenance
Broadcast tools for web chats	ChatHub, ChatALL	4 (web) when recorded and rerun; otherwise a level-1 habit	Individuals	One prompt, parallel answers, no reconciliation	Answers in panes; nothing on disk unless saved	Breaks when interfaces change; terms of service
Council as a product feature	Perplexity Model Council; Microsoft 365 Copilot research agent (GPT and Claude); Karpathy's LLM Council	4 (web, managed)	Subscribers of those services	Merged answer with agreement and divergence	Whatever the vendor shows; sources often folded in	Merged voice hides provenance; chooser is the vendor
Web chat exposed to agents	OpenCLI and similar	Bridge from 4 (web) to 4 (CLI)	Technical users	Agents can use web subscriptions	Files, if the agent writes them	Fragility; access rules

Pattern	Examples (September 2026)	Level	Built for	What it buys	Record it leaves	Main risk
Vendor plugin: one agent delegates to another	OpenAI's plugin for Claude Code; Claude and Codex inside GitHub Agent HQ (Copilot)	4 (CLI)	Developers, spreading	Cross-vendor review and delegation inside one session	Session transcript; review output	Mostly code-shaped; quota of two services
Council and consultation servers	A multi-provider consultation server (last commit December 2025); consensus plugins; small CLI councils that write one file per tool	4 (CLI)	Developers	Several families on one task; consensus rules	Per-tool files; a merged note	Rounds without end; correlated errors; abandonment
Multi-agent dashboards	Several official CLIs in panes, each with its own working copy, a person able to take over any of them	4 (CLI, visible)	Developers supervising parallel agents	Supervision made visible	Per-agent logs and diffs	Built for code; attention spread thin
Fan-out script with time ceilings	Listing C1 in Appendix C	4 (CLI)	Anyone who owns a level-3 agent	Parallel answers kept even when a tool stalls	One file per tool, a summary with who answered	Needs a person to reconcile
Single agent, task written down and rerun	An instruction file or skill run on a schedule	3 made repeatable	Anyone at level 3	Repetition where a second model is unavailable	Task file, inputs, outputs, acceptance note	Repetition amplifies a bad template

The rows describe patterns. The star counts in Appendix E are a popularity trace on one day, and they are not users, quality, or fitness for a folder of contracts. Abandonment belongs in the table: the best-known consultation server last committed in December 2025, and more than one dashboard has had its maintainers wind down. A paper that pointed learners at a single named orchestrator would be doing them a disservice, which is why I point them at the pattern instead: official binaries, non-interactive flags, files on disk, and a record of who was asked.

7.2 Why this layer lasts

CLI orchestration rests on the vendors' own tools, run in the modes the vendors themselves document for automation, and that gives it properties a browser tab cannot have. Above all, it can be written down. An arrangement that worked once, say "draft with one model, have a second review against the checklist, give me the disagreements", can be saved as an instruction file or a skill and run again next week, on a schedule if need be. Section 3 defined level 4 as exactly this, repeatable work directed across models. Several further properties follow, in the order I weigh them. A command-line flag is a documented interface and an exit code is a promise; a position on a web page is neither, which is why tools that drive consumer chat pages have to be repaired whenever the page changes. Each official tool runs under its own vendor's login, with that vendor's sandbox and deny rules, so the person can keep secrets out of the folder, deny network by default and refuse destructive commands per tool, whereas a browser agent that can see every tab is one larger permission object. The pattern uses consumer logins the person already pays for, and I expect that, more than anything, to separate a practice a non-programmer keeps from one they try once. And the vendors bless it. The official plugin, the agents inside GitHub's service, the headless modes and the skills standard are signs that cross-tool delegation is being standardised, while the automation of consumer chat pages is being chased off.

Web orchestration may teach the habit of comparison, and CLI orchestration is where the habit can be kept, as an object. The object is a folder that holds the assignment, the permitted inputs, the outputs, a log of which agent produced what, and a note of what was accepted. Another person can inspect it without replaying a private chat, and the person who made it can change one input next week and see the difference. That record also makes responsibility harder to evade, since a saved assignment and an acceptance note expose a poor source set or a missing approval. In offices used to informal chat it will feel like a burden, and wherever work has to be checked, handed over or repeated it is an asset. Proportionality decides which: a two-minute request does not need a workflow folder, while a recurring task that touches shared files may.

Most of what exists is shaped by code. The official plugins, the council servers and the multi-agent managers target software review, although nothing in the pattern itself is specific to code.

7.3 What the layer asks of the person

A small team of agents needs a manager, and the manager's work is specific. I call its four parts the delegation contract:

1. Goal: what the work is for, stated so that an agent could tell whether it has been met.
2. Boundary: what the agents may touch and what they may not, which actions need a person's confirmation, and what budget in time and quota the task may spend.
3. Artefact: what should exist when the work is done, and in what form.

4. Check: how the person will know the artefact is right, decided before the work starts, who reviews what, and when to stop.

The fourth part carries a trap described in the companion paper on automated oversight (Chernets, 2026b). Adding a reviewer is cheap, and every reviewer finds something. A council without a stopping rule tends to add rounds and critics until the work is buried in review. A practical rule is to allow one round of critique and one round of fixes per stage, with the person deciding whether a finding blocks the work. The rule has a measured price: in a randomised test of 144 runs reported in that paper, a one-round budget cut critique rounds from 2.86 to 1.00 per run while raising missed defects by 0.11 per run (95% confidence interval 0.005 to 0.204) (Chernets, 2026b). Whether that trade is worth making depends on the stakes, which is exactly the kind of decision that belongs to the manager. Limits on rounds and time are budgets, and a budget settles when to stop, which is a different question from whether the answer is true.

The contract restates what good delegation to people has always required, for an audience that has never delegated to software, and it claims no new theory of management. Its value is that each part can be taught, written down and checked, and that each part fails in a way that can be seen when it is missing.

8. From operator to manager

8.1 What changes in the job

At level 1 the person produces and the model assists. At levels 3 and 4 the model produces and the person directs, supervises and accepts. Bainbridge's (1983) ironies of automation predicted what this does to the human role. Automation removes the routine and leaves the person the residue that is hardest to do well, monitoring and intervening in rare failures, while the practice that built their judgement disappears. The generative-AI version of the irony is already documented. Generative AI shifts people from producing to evaluating and makes hard tasks harder (Simkute et al., 2024), higher confidence in the tool goes with less critical thinking (Lee et al., 2025), and deskilling follows structurally from the removal of practice (Ferdman, 2025).

Managing an agent is therefore a different job with its own competence, and the competence does not come free with the tool. It consists in choosing what to delegate, stating the delegation contract of Section 7.3, reading an agent's trace, and deciding when the result is good enough. I break the job into four practices. Delegation names the work as a deliverable with a boundary. Supervision watches the plan, the intermediate output or the tool trace and intervenes when the task drifts, without approving every keystroke, because approval of everything is fatigue and fatigue is how people click yes. Verification checks sources, sums, transformations and the claims a decision rests on, in proportion to the risk and by a method other than rereading the agent's own confidence: tests and diffs for code, a spot check against a page the person opens themselves for a folder of contracts, and for a literature table a row-by-row check of identifiers. Revocation narrows the workspace, stops a run or removes access when a condition changes. A person who

does all four is managing an agent even through a graphical desktop application. A person who hands a task to an opaque service and accepts the first answer is outsourcing judgement, which is a different practice. Parasuraman and Riley (1997) named the two ways to fail at it. Disuse is the person who could delegate and never does, stuck at level 1 by habit or fear. Misuse is the person who delegates without checks and accepts what comes back. Both are visible in 2026. The broad-and-shallow adoption data describe disuse at scale (Section 2.2), and the file-loss reports and database deletions of Section 5.3 describe misuse, sometimes by professionals who knew better. Table 6 sets each practice beside the part of the delegation contract it enforces, the risk it meets and the dated trace by which a study can see that it happened.

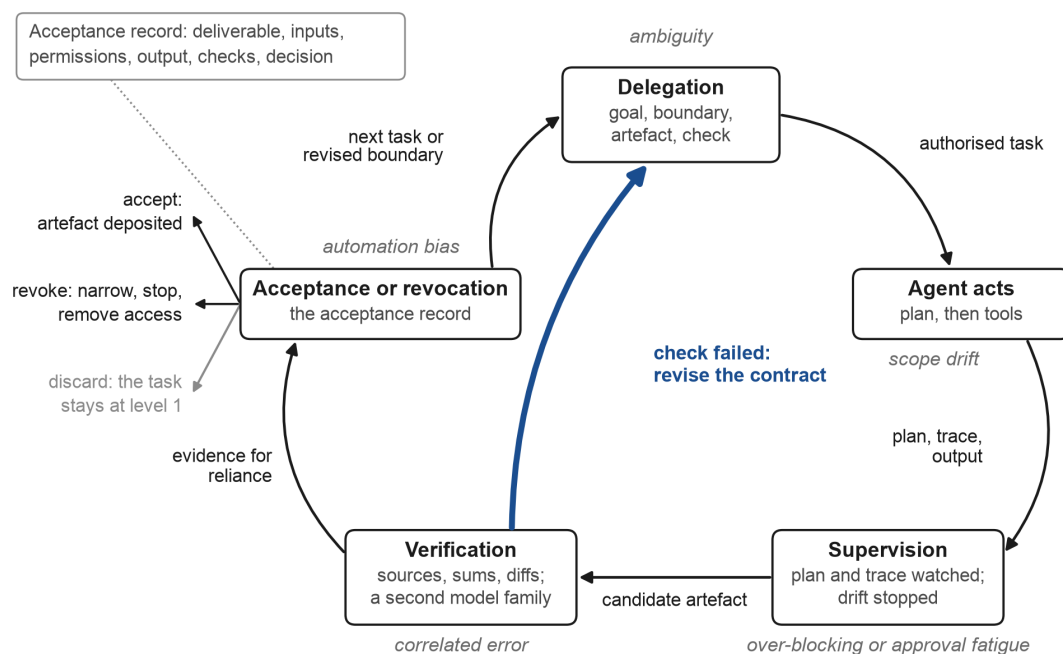
Table 6. *The manager's four practices: what each enforces, the risk it meets, and the trace it leaves.*

Practice	What the person does	Contract part it enforces	Risk if it is missing	Observable trace
Delegation	Names the work as a deliverable with a boundary (8.1)	Goal, stated so that an agent could tell whether it has been met; artefact: what should exist, and in what form (7.3)	Disuse: “the person who could delegate and never does”, stuck at level 1 by habit or fear (8.1)	An artefact deposited (Appendix A)
Supervision	Watches the plan, the intermediate output or the tool trace; intervenes when the task drifts, without approving every keystroke (8.1)	Boundary: what the agents may touch; which actions need a person's confirmation; what budget in time and quota (7.3)	Approval fatigue: “fatigue is how people click yes” (8.1); a guard that blocks too much produces disuse (5.3)	A plan refused; an irreversible step held for confirmation (Appendix A)
Verification	Checks sources, sums, transformations and the claims a decision rests on, by a method other than rereading the agent's own confidence (8.1)	Check: how the person will know the artefact is right, decided before the work starts; when to stop (7.3)	Misuse: “the person who delegates without checks and accepts what comes back” (8.1); correlated errors among models (6.3)	A diff or output rejected; a second model family asked; a denominator recorded: asked, answered, timed out (Appendix A)
Revocation	Narrows the workspace, stops a run or removes access when a condition changes (8.1)	Boundary, applied again: what is kept out, and how to undo a mistake (3.2, 7.3)	A long task that could not be stopped, after which deleting the task deleted the files (5.3)	None among the dated traces of Appendix A; a dated stop, narrowed permission or removed access would serve

Note: A person who hands a task to an opaque service and accepts the first answer is “outsourcing judgement”, which is a different practice (8.1).

One small device holds the role together across all four practices, and I call it an acceptance record. For a consequential or recurring task it holds the intended deliverable, the inputs used, the constraints and permissions, where the output went, any material disagreement or failure, the checks performed, and the person’s decision to accept, revise, escalate or discard. It draws a line between what the agent produced and what the person relied on. To the individual it is useful because it turns a feeling of trust into a checkable decision, to a team because a colleague can inspect the work without a private chat history, and to a researcher because it separates a completed run from a relied-on result. It is also a restraint. A person may decide, after failing to write an acceptance criterion, that the task should stay at level 1, and the model counts that as mature management rather than as lagging adoption. In the vocabulary of human-AI collaboration, the move from level 1 to level 4 keeps the human in the loop and changes what the loop is, from reading each answer to setting the terms on which answers are accepted. Figure 4 draws that loop with the four practices, the risk at each step, the return after a failed check and the acceptance record.

Figure 4. *The manager’s control loop.*



Note: The manager’s work as a loop the person runs (Sections 7.3 and 8.1): delegation states the contract (goal, boundary, artefact, check), the agent acts, the person supervises and verifies, and then accepts, revokes or discards the result and writes the acceptance record. The grey italic labels name the risk at each step (Sections 5.3, 6.3 and 8.1). The heavy arrow is the return from a failed check to a revised contract, and every exit from the acceptance node is a decision of the person.

8.2 Experienced users delegate less

One observation cuts against any simple story in which everyone should end up as a manager of agents. Anthropic reported in March 2026 that users with longer tenure were

“much less likely to delegate greater responsibility through directive use patterns” (Anthropic, 2026e), and its January 2026 report put augmentation, collaborative use in which the person stays in the loop, at 52% of conversations in November 2025 (Anthropic, 2026f). One reading is that experienced users learned where delegation fails and pulled back. Another is that they learned to use the model as a collaborator for the parts of work where their own judgement is the product. I cannot tell the two readings apart from the published figures, and on either reading the endpoint of the ladder is something other than maximal delegation. It is the ability to choose, task by task, between asking, collaborating and delegating, and to know which the task in front of you needs. The four-level model describes what a person *can* do. It makes no claim that they should do the most delegating thing every time.

8.3 Responsibility stays with the person

The vendors are clear about where accountability lies. The launch guidance for Anthropic’s level-2 agent told users that they “remain responsible for all actions taken by Claude performed on your behalf” (The Register, 2026). This is the legal and practical reality of level 3 in particular, where the agent acts with the owner’s credentials, on the owner’s files, under the owner’s name. A professional who becomes the manager of an agent also becomes accountable for a worker whose mistakes are fast. It is one more reason the check in the delegation contract has to be decided before the work, and one more reason the safety design of Section 5.3 should not depend on the person spotting an attack in real time.

8.4 The manager’s role in the literacy frameworks

Official definitions of AI literacy are beginning to name this role, although none of the texts reviewed here uses the word “agent”. The US Department of Labor’s AI Literacy Framework of February 2026 lists “directing AI effectively” among its content areas (US Department of Labor, 2026; ExecutiveGov, 2026). The joint OECD and European Commission framework of June 2026, which feeds the PISA 2029 assessment, has four domains, one of which is “Manage AI” (OECD & European Commission, 2026). Earlier academic frameworks stressed recognising, understanding and evaluating AI (Long & Magerko, 2020), and UNESCO’s 2024 competency frameworks for students and teachers set the same register (UNESCO, 2024a, 2024b). The delegation contract gives the managing and directing domains a teachable content for the agentic case: set a goal, fence the work, name the artefact, decide the check. Practitioners have started to use titles such as “AI operator” and “agent manager” for a professional who acquires this specialty on top of their profession. I use “manager of agents” more narrowly, for a pattern of behaviour rather than a role anyone should hire against, because the behaviour leaves evidence and a title does not. The evidence is in artefacts: a plan the person refused, a diff they rejected, a second opinion they took, a scheduled run that finished while they were away from the keyboard. Where none of those traces exists, the title has nothing behind it.

9. Measuring the crossings: a preregistered programme

9.1 Crossings as the unit of measurement

Most adoption data count people who “use AI”, sometimes weekly or daily. A count of that kind cannot tell a person who asks a chatbot for synonyms from a person who delegates a quarterly report to an agent, and it is the second person I want to find. The artefact rule of Section 3.3 makes the difference observable, since a crossing is recorded only when the agent has produced something the person asked for, at the new level. Several further rules follow from the literature on onboarding and oversight. Completion should be reported separately for people who crossed unaided and people who were helped, because help changes what a crossing means. A stop should be recorded at the point where it happened and in the person’s own words, because the reasons for stopping are a large part of the result. Every person who received the materials belongs in the denominator, including those who never replied. Persistence, finally, should be observed after the researcher has stopped writing every week; an artefact produced while the researcher is still in a person’s inbox may be courtesy as much as use.

9.2 The design

I run a preregistered cohort programme that applies these rules to non-programmers (Chernets, 2026d; OSF, <https://osf.io/x4egq>, DOI 10.17605/OSF.IO/X4EGQ, registered and frozen on 6 September 2026). It is a single-arm feasibility study of a three-step pathway, and the preregistration was frozen before any data were collected. What follows is the design as registered.

Participants. Adults whose work is substantially at a computer. They use their own machine and their own AI subscriptions (an employer’s are excluded), keep client-identifying and employer-confidential material out of the exercise, and confirm that installing software does not breach a rule their employer applies to them. Recruitment is by public announcement on professional social networks within a window fixed in advance by time, so that it cannot be extended until a result looks favourable. The announcement is screenshotted and its permanent link recorded, which fixes the recruiting text. Entry is rolling and each participant’s schedule counts from their own day zero, because the tools change on a weekly scale. Recruitment opened with the public announcement on 6 September 2026, the day the registration was frozen, and closes 21 days later, on 27 September 2026; the cohort is in the field at the time of writing, and the realised number, whatever it turns out to be, will be reported with the results.

Step 1 (from chat to an agent on the person’s own computer). The participant pastes a standard text into the chatbot they already use, and that chatbot guides them through installing a graphical agent on their own machine. I do not touch anyone’s computer. The step is complete when the participant supplies an artefact the agent produced in response to a first task. The task is identical in shape for everyone and filled with the participant’s own working situation, and it requires the agent to ask, decide, create and format files.

That requirement separates an agent from a chatbot. The registration gives the exact wording (Chernets, 2026d). In the terms of the model, Step 1 tests reflexive onboarding across two levels at once, from level 1 to level 3. That was a deliberate choice. A graphical shell of a local agent runs on the person's own machine, so by the rule of Section 3.3 it is level 3, and the artefact Step 1 asks for is the kind that marks transition 2; its permission prompts give it some of the protection of a scoped rung. The cohort does not instrument a separate week in a vendor-fenced level-2 environment, so it cannot test P5 of Section 10 (that a scoped middle rung lowers non-completion and incidents); those permission prompts are the only scoped rung the design has, and the command line enters at Step 3. A later cohort that inserts an explicit level-2 week between chat and a local install would be the clean test of P5.

Step 2 (into web orchestration, launched from the agent). Once ready, participants receive an open-source tool that puts one question to several models through the subscriptions they already pay for and returns one merged answer, in a single action from inside the agent. The purpose of the step was registered in advance as coverage and reliability, with visible disagreement as one mechanism for the second.

Step 3 (into CLI orchestration). The same orchestration through command-line interfaces.

Measures. Five frozen core items, worded identically in every cohort of the programme, record profession and hours at a computer, current AI use, prior experience of asking more than one AI, whether and how answers are checked, and whether anything already acts for the person on their computer. A step record for every participant holds the dates, the chatbot that guided them, attempts and hours, completion with the artefact, the exact stopping point and the reason in their own words, and whether and how the investigator intervened. Weekly for the first month, participants report one artefact their agent produced that week (or “nothing”, which is a valid answer), how often they put one question to more than one AI and what happened when answers diverged, and from Step 2 onward whether anything useful came up that one model had missed and whether checking changed what they relied on. Observations continue at day 60 and quarterly after that, for as long as participants are willing.

Analysis and interpretation. The analysis is descriptive. Completion proportions are given with confidence intervals against both the surviving and the full denominator, separately for unaided and assisted completion; times and attempts are reported as distributions, and reasons for stopping are coded after collection, with codes and raw wording deposited. One interpretation rule was fixed in advance. If fewer than half of those who received the Step 1 materials supply an agent-produced artefact, the cohort is reported as a study of onboarding failure, and what stopped people becomes the result.

Limits of inference. The registration states in advance that there is no control condition and therefore no causal claim, and no generalisation beyond self-selected volunteers. A participant's estimate of time saved is reported as belief only, because self-report in this domain has been shown to invert the true direction (Becker et al., 2025).

Disclosure and ethics, as registered. Participants are told in writing that I wrote the materials and the tool, teach them, and am an applicant on pending patent applications covering the orchestration method. No institutional review board opinion was sought: there is no institutional affiliation behind the study and no federal funding. The registration states the safeguards instead. Consent is written and given before anything is recorded. No health, financial or work-content data are collected. Participants work on their own machines and subscriptions and may ask for deletion at any point up to public deposit. The safety instructions include a separate browser profile, no actions on banking or medical sites, confirmation before anything irreversible, no passwords typed into the agent, and web content treated as data. No disability variable is collected and no accessibility claim is made.

9.3 The design only

From the programme I take the design and nothing else. No participant numbers, completion rates or individual cases appear here, by deliberate decision: the argument (this paper), the method and programme, and the results after the protocol's observation period are separate publications, so that the argument cannot be read as evidence for itself. Other researchers can apply the same rules now. Listing C5 in Appendix C gives a record format and a checker that encode them. The checker refuses a record that counts an installation as a crossing, a completion without an agent-produced artefact, a stop without the person's own words, a real name in place of a pseudonymous label, a field the format does not know, or a skipped level that is not recorded exactly as skipped.

9.4 What a strong next study looks like

The cohort is a feasibility study, and most of the propositions of Section 10 need other designs. A strong next study would compare two onboarding conditions that differ in how they represent the work, holding task risk and the technical environment as constant as possible. One condition presents features and installation instructions; the other begins with the participant's own recurring work object and elicits the five-part map of Section 5.1. Outcomes would include artefacts, their quality rated by someone other than the participant (with agreement between raters reported, as for the coding of stop reasons), time, errors discovered, permission decisions in scenario form, later voluntary reuse, and confidence measured against results. A follow-up would separate persistence from continued contact with the researcher. A second study would ask whether workflow vocabulary is one grammar or one per profession, by comparing administrative, research, communications and analytical tasks. I do not know which it is, and the answer decides whether packaged examples can be written once or must be written for each field. Both studies should baseline AI literacy with a validated instrument (Carolus et al., 2023), so that the specific gap, knowing what to hand to an agent, can be separated from general familiarity with AI. Every such study should also document the implementation, because a participant who starts on a later interface meets different defaults: the interface version, the operating system, whether the agent had network access, the permission mode, and any policy constraint. The vendor's name alone tells a later reader too little.

10. Propositions

The model of Section 3 and the argument of Sections 4 to 8 imply claims that can fail. Table 7 states fifteen of them, each with a measure, a result that would falsify it, and who is placed to run the test. They are written for the field as a whole. The programme of Section 9, for one, can test only a few; several others can be tested with vendor telemetry, with survey panels or with a small study in one organisation. None is a finding of the cohort.

Table 7. *Propositions, measures, falsifiers, and who can test them.*

#	Proposition	Measure	Falsified if	Who can run it
P1	Barrier migration. Among non-programmers who attempt transition 2, stops cluster after installation (at the first task, the first error, or permissions) more than during it, and among those who installed and produced nothing, “I did not know what to ask” outnumbers technical, permission and cost reasons; and, task difficulty held constant, completion falls more sharply at the three transitions than within a level.	Stop points recorded with the artefact rule; reasons in the person’s own words, coded by the rule of Appendix A; completion by step	Most stops occur before the agent first runs, or technical and permission reasons dominate among installed non-producers, or completion and failure patterns are comparable across all steps once task difficulty is controlled	The programme of Section 9; any replication
P2	Surface merging. Where vendors merge chat and agent surfaces, the share of sessions that end in an agent-produced file or action rises without any change in what users say they want to do; and the leading assistants file-write from the default window without a user-chosen mode by the end of 2027.	Vendor telemetry before and after a merge; a dated census of the assistants	The share stays flat after a merge; or a stable, user-chosen agent mode remains a prerequisite for writing files	Vendors; anyone who can open the applications
P3	Vocabulary over installation. Giving people worked examples from their own field (packaged tasks, skills, templates) raises persistence at levels 2 and 3 more than giving installation help, and a structured measure of workflow vocabulary predicts artefact completion after access and prior AI use are controlled.	Day-60 persistence under the two kinds of help, in a comparison with assignment; a pre-task statement of the five parts of Section 5.1; stop reasons coded by the rule of Appendix A	Installation help produces equal or greater persistence; the vocabulary measure adds nothing once installation ease and generic confidence are in the model	Any laboratory with two onboarding routes

#	Proposition	Measure	Falsified if	Who can run it
P4	Reflexive onboarding. People guided by the chatbot they already use can cross from level 1 to level 3 unaided at a rate that makes the route practical, and reach a first artefact sooner than people given static documentation.	Proportion completing transition 2 unaided with an agent-produced artefact, against all who received the materials; time to first artefact under guided and static conditions	Fewer than half complete it, the threshold fixed in advance in Section 9; or static documentation gives equal or higher completion	The programme of Section 9; trainers with two cohorts
P5	The middle rung. Non-programmers who spend time in a vendor-fenced environment before a local agent complete transition 2 more often, have fewer file-loss incidents, and reason better about scope, reversibility and escalation than people moved straight from chat to broad local access.	Two-route cohort with equal follow-up; artefact as the primary outcome; an incident log; scenario-based permission decisions before and after	Unaided completions from chat straight to a local agent match those that passed through level 2, at comparable incident rates and with equal permission decisions	Any laboratory with two onboarding routes; a later cohort of the programme
P6	The bottleneck. Among non-programmers who use chat weekly, the share producing a level-3 artefact in a quarter remains far below the share of developers who use coding agents weekly.	A representative survey using artefact items rather than “do you use AI” (Section 11.3)	Level-3 artefact production among non-programmer knowledge workers approaches the developers’ weekly-use figure	National statistical offices; survey houses
P7	The bridge carries. People who practise web orchestration are more likely later to adopt CLI orchestration than people who do not, and visible disagreement predicts later source attribution or critique in their task records.	Adoption of CLI orchestration among those with and without earlier web comparison; saved comparisons and later records	No difference, or the reverse; comparison produces more prompts and no change in retained checks	The programme of Section 9; longitudinal panels

#	Proposition	Measure	Falsified if	Who can run it
P8	CLI durability. For people who pay for two or more consumer subscriptions, orchestration through official command-line tools is the form of level 4 that survives interface changes, and file-backed runs rerun and transfer to another reviewer better than browser-only runs.	A dated census of level-4 artefacts in that population; rerun and transfer tests after an interface change	A late-2027 census finds web or programming-interface councils dominant and CLI-to-CLI tools receding; browser-only workflows match on rerun and transfer	Tooling researchers; developer surveys
P9	Correlated agreement. Across vendor families, errors on knowledge-work items with known answers correlate enough that agreement among three or more models is worth fewer than two independent opinions (rho at or above about 0.5, where rho is estimated on knowledge-work items with known answers and not taken from benchmark agreement rates); a small council of different families beats a large council of one family at equal cost; and source-aware review predicts correctness better than majority agreement.	Pairwise error correlation across families on item banks with known answers (listing C4); a marked item set with two council designs; correctness against majority agreement and against source-aware review	Measured rho below 0.2 for most task types; the same-family council wins after a claim-by-claim check; or simple majority agreement predicts correctness as well as or better than diversity plus verification	Language-model researchers; the companion paper's protocol
P10	The manager's competence. Persistence at levels 3 and 4 is predicted better by a person's ability to state the delegation contract (goal, boundary, artefact, check) and by supervision traces (plans refused, diffs rejected, second opinions taken) than by prior technical experience or by prompt volume.	A short task that elicits the four parts, scored blind; baseline items; traces from the acceptance record; persistence at day 60	Prior technical experience or prompt volume predicts persistence as well or better, and contract-stating adds nothing; or artefacts at levels 3 and 4 that routinely omit goal, sources, boundary and check remain dependable under independent audit, in which case the competency structure is overbuilt	Any longitudinal artefact study
P11	Calibration by task class. People calibrate delegation better where feedback is fast and reversal is possible, accepting, correcting and escalating differently by task class.	Acceptance, correction, escalation and later error discovery by task class	Calibration is uniform across reversible and high-consequence tasks	Laboratories; organisations with an acceptance record

#	Proposition	Measure	Falsified if	Who can run it
P12	Friction from guards. Among non-programmers at level 3, interruptions by permission prompts and safety blocks are named as a reason for stopping more often than among developers, and classifier-guarded autonomy reduces such stops without an increase in reported loss of files.	Coded stop reasons by group; stop rates and incident reports before and after a change in default mode	No difference between groups, or autonomy raises reported losses	Vendors; the programme's coded stop reasons
P13	The gap reproduces. The differences in chat adoption by gender and education (Cranney et al., 2026; Daepf & Counts, 2024) reappear at least as large in level-3 artefact production within occupations.	Within-occupation differences in level-3 artefact production	A within-occupation estimate shows the gap closed at level 3	Labour economists; later cohorts if large enough
P14	Critic and second family. A same-family critic catches haste and skipped checks; a second family catches shared error; on a defect-labelled set each contributes defects the other misses.	Overlap of confirmed findings between a same-family critic and a second-family reviewer at equal cost	Complete overlap, or one source dominating	Anyone with two agent tools and a labelled set
P15	Capability or vocabulary. At equal task class, persistence at levels 2 and 3 varies with a person's ability to state the delegation contract; if model capability were the binding constraint, persistence would vary between task classes and be uniform across persons within one.	Persistence by task class, crossed with a blind-scored statement of the delegation contract	Within a task class, persistence is uniform across persons whatever their contract score, and varies only between classes	Any study that records task class and the contract statement

A few of them matter more than the rest. P4 carries the heaviest weight for policy, because if reflexive onboarding works, the most expensive part of teaching people agents (sitting next to them during installation) can be replaced by the tool they already have. P5 is the one the cohort cannot test, as Section 9.2 says, and it is also the one on which the shape of the ladder most depends. Whether the middle rung earns its place is something I do not yet know. P9 is the easiest to test and the most likely to surprise users of councils, since the arithmetic of correlated evidence (Chernets, 2026a) and the measurements of Kim et al. (2025) both point toward a ceiling on what agreement is worth. P10 is the one that would change curricula. If the ability to state a delegation contract predicts persistence better than technical background, then training should teach delegation first and tools second.

11. Implications

11.1 For vendors

Design for the blank folder. Vendors have simplified the installer, and it shows. I think a further round of that effort would be wasted, because the newcomer's problem now begins after the installer, in front of an empty folder with nothing to say to the agent. Worked examples from the person's own field, offered the moment they first open the agent, together with a first task that is real, low in risk and ends in something they can judge, would do more for adoption than any further simplification of setup. Packaged skills already point this way (Section 5.1). Two things are still missing: help in choosing which of a person's tasks is worth handing over at all, and a way for a professional association, and not only a start-up, to write skills for its members.

Whatever the tabs are called after the next merge, a person needs a plain answer, at the moment of delegation and in the language of the selected folder, to a short list of questions: what can this agent read, what can it change, what can it send or trigger, what will need my approval, and how do I stop it? A visual diff answers part of this when files change, and a written plan does when actions are diverse; one consent banner answers none of it. The permission model should stay visible, scoped and reversible after the merge, with an undo and a stop button that works. Folder scope without either is how the issue reports of Section 5.3 read.

The work should also be able to leave the product. A user should be able to export the task instruction, the inputs or their identifiers, the tool actions, the output and the approval history. This is a usability feature as much as a governance one, because it lets the newcomer learn from a completed task and lets a team review a result without lending its memory to a proprietary chat history. Usage should be reported the same way. "Weekly active users" of an agent that produces files ought to be split into people who produced a file and people who opened the application, and vendors have the telemetry to do it.

Detecting attacks is not a job for the novice. Guidance that asks non-programmers to recognise signs of prompt injection puts the hardest judgement on the person least able to make it (Section 5.3). Safe defaults (working copies, snapshots before larger changes, destructive commands denied until explicitly allowed, web content treated as data) belong to the product. At the same time, guards that interrupt constantly stop work as surely as missing guards lose files, and the companion paper on oversight overreach describes how such layers grow (Chernets, 2026b).

Provenance should survive the merging of answers. Council features that return one merged answer should show which model said what and where they disagreed, let the user assign one model the role of critic, and make an evidence gap easier to find than a fluent consensus paragraph. A merged voice without sources restores the authority of a single confident answer, rewards automation bias and hides the correlation of errors (Section 6.3).

Churn, finally, has a cost on the user's side. A person who learned a tool that was renamed, merged or retired within a year pays twice, so stable names, migration guides and continuity of instruction files across product changes are adoption features in their own right.

11.2 For employers and trainers

Teach the delegation contract before the menus. Product menus change monthly, while the ability to state a goal, fence the work, name the artefact and decide the check transfers across tools (Sections 4.3 and 7.3). A short exercise that asks a person to write the four parts for one of their own recurring tasks tests readiness better than a tour of features does. What the exercise contains is set out below, under decomposition.

Logins measure the wrong thing. An active-user count cannot make the distinction that Section 9.1 needed, between the person who asks for synonyms and the person who delegates a report. A count of agent-produced artefacts that people chose to keep can make it.

People also need a sanctioned place to learn and a map of what is allowed. Level 2 inside tools the organisation already licenses reaches staff who will never be allowed to install an agent on a work laptop (Section 5.4). For level 3, a clear policy, a sandboxed machine or an approved configuration removes the silent “no” that many people assume. A blanket ban drives use through personal accounts where nobody can see it, and unrestricted delegation exposes confidential material. Between the two lies a task and data taxonomy. It says which work may be tried in approved environments, which needs a second person's review and which is excluded from agents; it names who may approve a tool, where the acceptance record lives, and how a near miss is reported without punishment for reporting it. Local level 3 should not be required on machines the employee does not control, since cloud agents and tenant-scoped level 2 are the path that avoids a fight with the IT department. Once the vendors absorb the first rung into chat, level 2 keeps two jobs. It is the sanctioned place for staff on locked machines, and it is the calibration rung for everyone else, where a person learns that delegated action has a perimeter before owning one. Whether it also lowers incidents is P5.

Mandates need care. Making AI use a performance criterion can produce use without value, and at least one employer that did so reversed the decision within a year (Fortune, 2026b). Asking for artefacts that matter to the job is a better signal than asking for use. Once a month, “show a file an agent produced in a folder you named” and “show a decision you changed because two models disagreed” measures transitions 2 and 3, which a login count cannot. Training at scale without artefact measures, however many thousands are enrolled, will not tell anyone whether transition 2 happened.

Decomposition and verification should be taught together, and the critic habit before the council. A generic prompt course is level-1 pedagogy. Level-2 and level-3 pedagogy starts from the job the person already has, a marketer's Friday pack, say, or a teacher's batch of assignments, teaches a reusable decomposition (the five parts of Section 5.1, with a

condition for stopping) and teaches error recovery on a copy of the folder. What to assess afterwards is whether the person can recognise an unsafe instruction, name a reversible trial, inspect the evidence and stop the system. A same-family critic at acceptance points costs nothing in subscriptions, and a second family is added when the critic is silent and the decision is expensive. Orchestration at the top of a fluency rubric is the right destination for a few roles and the wrong day-one requirement for everyone.

11.3 For policy

The European Union's AI literacy duty applies to every provider and deployer of AI systems for their staff. After the 2026 amendments it requires them to "take measures to support the development of AI literacy" rather than to ensure a sufficient level, and its supervision and enforcement rules apply from 3 August 2026 (European Union, 2026; European Commission, 2026a, 2026b). In the United States the Department of Labor's framework names "directing AI effectively" (US Department of Labor, 2026), and the OECD and European Commission framework has a domain called "Manage AI" (OECD & European Commission, 2026). I suggest a content for those words in the agentic case: the four parts of the delegation contract, the four practices of Section 8.1 including revocation, the habit of comparing models with the knowledge that agreement is weaker evidence than it looks, and the basic safety practices of Section 5.3. Guidance should distinguish conversational use from delegated action. For the latter, the duty should cover permission scope, data classification, verification, incident reporting and revocation, and a requirement to document the role-specific capability and authority needed for delegated tasks is more defensible than a requirement to train everyone identically. Regulators and worker representatives can also ask whether an employer has moved residual responsibility onto staff without usable controls, time to review, or a right to decline an unsafe delegation. All of this is a design and governance recommendation drawn from one regulatory instrument, and its scope is that instrument.

The European duty survived a deregulatory round, and among the frameworks discussed here it is the one horizontal obligation that covers every member of staff who uses AI. In the United States the frame is workforce policy. An executive order of April 2025 on AI education and the July 2025 AI Action Plan commit to expanding AI literacy and skills, including guidance that AI literacy programmes may qualify as employer-provided educational assistance (White House, 2025a, 2025b); the frame is compatible with training managers of agents and does not yet require it. None of the official texts inspected names terminal agents or orchestration, so "Manage AI" can be read as chatbot etiquette or as level-4 direction. Literacy programmes that stop at prompting teach level 1. The work, meanwhile, is moving to levels 2 to 4. The official texts have not yet followed it there.

Statistical agencies adding AI modules to their surveys could help with one specific change. "Have you used a chatbot" is a level-1 item. "Has a system you directed created or changed files on a computer you use for work in the last seven days" is a level-2 or level-3 item. "Have you put the same work question to more than one AI system in the last seven days, and did the answers differ" is a level-4 item. Without items of this kind, P6 cannot be

tested in public data, and the second-level digital divide at the agentic tier will stay invisible to the people whose job is to see it.

11.4 For researchers

The gaps of Section 2.5 are an agenda. Study non-programmers using terminal and desktop agents by name, with artefacts as outcomes. Study reflexive onboarding as a mechanism in its own right. Study web orchestration as a practice, including how people reconcile answers and whether they keep provenance. Measure the correlation of errors across vendor families on knowledge-work tasks, and on more than benchmarks of code and mathematics. Extend the measured gender and occupational gaps in AI use (Cranney et al., 2026; Humlum & Vestergaard, 2025) to the agentic levels, where nothing is yet known (P13). Two easy metrics, feature availability and self-reported time saved, are worth resisting in favour of artefact completion, audited quality, correction rates and later reuse. Abandonment should be recorded with its mechanism. A stop at a permission prompt may be an interface problem, a sensible worry about data, an employer rule, a missing machine or a task with no acceptance criterion, and collapsing them into “resistance” loses all of that. Evaluation should be adversarial, with a misleading page, a hidden instruction in a document, a contradictory source, a partial tool failure, or a convincing wrong answer that several models share, because the manager’s role means something only if it improves outcomes under those conditions. Replication needs none of my materials; it needs the artefact rule, a frozen stop-reason question, a record of the implementation, and a dated product census. And the denominators should be published, since a study that reports only those who crossed tells us nothing about the road.

12. Scope and boundary conditions

What is offered here is a model, an evidence synthesis and a measurement design, and each is bounded in the following ways.

Time. The evidence runs to 21 September 2026. Product facts are dated and will age. The model is defined by reach and role so that it ages more slowly.

Geography and sources. Most adoption figures are from the United States; European figures are cited where they exist. Several key figures come from vendors (usage of coding agents, the reasons for merging chat and agent, injection rates on vendor test sets) and are marked as vendor data in the text. I use them for what vendors can know about their own products and weigh them accordingly.

Population. The argument concerns professionals whose work is substantially at a computer. People who work mainly from a phone, or without a computer they control, meet a barrier before any described here, and the model says little about them beyond naming that barrier (Section 5.5).

Evidence about the cohort. Only the preregistered design is used. No outcome of the programme informs any claim here, and the propositions of Section 10 are stated before

its results are known, so that they can be judged against them. A cohort that requires a machine of one's own samples people who are able to install an agent; it cannot separate equipment access from motivation, and that limit travels with the results.

Normative scope. The model describes what each level asks of a person. It does not hold that higher levels are better or that everyone should climb; for much work, level 1 or 2 is the right place (Section 8.2). High-consequence work, confidential records, safety-critical operations, time-critical decisions and tasks with no inspectable acceptance criterion need stricter arrangements, or stay outside delegated agent use altogether.

What orchestration adds. Adding agents can widen coverage, expose disagreement and leave a review trail. It can also add correlated error, cost, delay and a false look of diligence. Delegation that lasts means a person can inspect and alter the process; whether the process merits reliance is a separate question.

What the literature lacks. The literature search of Section 2.5 found no field study of non-programmers adopting terminal or desktop agents by name, no study of web orchestration as a named practice, no treatment of managing agents as a competency with its own structure, and no study of an AI system scaffolding a person's adoption of agentic AI. Those are gaps in what has been studied, stated as of 21 September 2026, and I claim nothing about markets, products or priority.

Status of claims. Table 8 applies the series' confidence language to the elements of the argument, so that a reader can see how firmly each is held.

Table 8. *Status of the claims in this paper.*

Element	How it is said here	Basis
Four-level model and the artefact rule	Being tested (Sections 9 and 10)	Conceptual; markers specified in Section 3 and Appendix A
The technical barrier has receded for installation and the desktop face	The data support it	One-step installers; "No terminal required" documentation; four vendors' level-2 products shipped
Knowledge workers inside a terminal-heritage agent	Vendor-reported, directional	Press report of vendor figures, 2 June 2026, with an analyst's caveat
Vendors absorbing transition 1 into chat	Observed in product	Anthropic, 16 September 2026; OpenAI, 9 July 2026
Workflow vocabulary as the dominant remaining barrier	Offered as a model; being tested	Indirect: prompting studies; the vendors' stated reason for merging; the stop-reason item of the programme
Web as bridge, CLI as durable layer	Being tested (P7, P8)	Contracts, permission objects, official plugins; fragility of web interfaces

Element	How it is said here	Basis
Correlated errors cap what a council buys	The data support it on model evaluations; being measured in use	Kim et al. (2025); Chernets (2026a)
Reflexive onboarding	Implemented and being measured; no outcome reported	Vendor and course practice (observation); Step 1 of the programme (measurement)
The manager of agents as a role	Offered as a description of behaviour; being tested (P10)	Delegation studies; employer rubrics; official literacy frameworks
Cohort crossing rates	Not reported	Protocol frozen; results paper to follow
“Works now” as a promise	Not claimed	Outside this paper’s scope

13. Conclusion

The terminal used to be the obvious picture of the barrier between ordinary professionals and agentic AI, and for a while it was a fair one. In 2026 it no longer fits. Installation takes a command or a click, the chatbot a person already uses can guide them through it, and the vendors are merging chat and agent into a single surface. What Sections 4 and 5 show is a barrier that has moved. The first bounded attempt has become much easier to make; what remains hard is getting reliable work on whole tasks, keeping up with products that change under one’s feet, and keeping a granted folder safe. On one vendor’s figures, reported through the press and to be read as directional, about a million knowledge workers a week are already inside a terminal-heritage coding agent. Most professionals, meanwhile, still meet AI only as a conversation.

What stands in the way now is the manager’s work: knowing what to hand over, fencing it, naming the result and deciding how to check it, then running an environment safely and comparing models while remembering that their agreement is weaker evidence than it looks. The four levels of Section 3 give that work a structure that no product owns, and the artefact rule gives it a unit of measurement. I have proposed web orchestration as the bridge to the last level (P7) and CLI orchestration as the form of it that lasts (P8); both are stated so that they can fail.

For anyone introducing the transition, the practical consequence is an order of operations. The first question is which small, permitted piece of work a person can describe, inspect and safely discard if the attempt fails: a briefing from public sources, a personal planning folder, a spreadsheet nobody else depends on. From there, one managerial responsibility is added at a time, a boundary, then an approval, then a workspace of one’s own, then a saved task record, then a second and critical view. Evaluation follows the same order. A demonstration is worth little if it hides what the person was allowed to do, how the output was checked and what happened when the agent failed. A considered refusal to delegate counts as a result. And a person who can reproduce a bounded task, say why they relied

on it, point to a disagreement that changed a decision and narrow access when circumstances shift is showing the observable signs of the role, whatever their job title says.

The model can fail, and Section 10 says how. The programme of Section 9 will test part of it on people who are not programmers, and the code in Appendix C lets others measure the same crossings and practise the same patterns. I expect the products named in Appendix B to look dated within a year. The barriers of Section 5 should not, because they belong to the person and the work, whichever product is in front of them. The terminal has become a place where an agent works for a person who has learned to manage it, and how many people will learn that, by what route and how fast, is the question the next year will answer.

Declarations

Funding. No external funding supported this work.

Competing interests. The author develops multi-model orchestration methods and has filed related patent applications (pending; no patent has been granted). The author designed the preregistered cohort programme described in Section 9 and its materials. No finding in this paper depends on the author's implementation; every pattern in Appendix C is reproducible with publicly available tools. The author also maintains the two open-source plugins named in Appendix C; they are free and open source (Apache License 2.0), and no finding in this paper depends on them. The author is developing teaching materials that may later be offered commercially; nothing has been sold at the time of writing, and the cohort described in Section 9 received all materials free of charge.

Tools and verification. AI assistants were used as instruments under the author's direction; the ideas, research and conclusions are the author's. All quantitative claims and citations were checked against primary sources or archived copies as of 21 September 2026 or, where a source is marked secondary or vendor in the text and in the timeline of Appendix B, against the best available report; figures reported by vendors about their own products are marked as such. Where a primary page could not be retrieved during the research window (several pages on one vendor's site returned an access error), the claim is cited from a named secondary source and labelled, or it is omitted. Press titles given in square brackets in the references are descriptive titles for pages whose headline was not re-read verbatim.

Data and code availability. The code in Appendix C, with its tests, is deposited under the MIT licence at <https://github.com/vadimchernets/after-chat> and archived on Zenodo, <https://doi.org/10.5281/zenodo.22944451>. Two plugins built on the same patterns, Roundcall and Sidecall, are at <https://github.com/vadimchernets/roundcall> and <https://github.com/vadimchernets/sidecall> (Apache License 2.0). Five kits that apply the same patterns to trades outside software are at <https://github.com/vadimchernets/calling-kits> (MIT). They have not been tried by practitioners of those trades, which the repository states, and no claim in this paper rests on them.

Ethics. This paper reports no data from human participants. The cohort programme it describes is preregistered (OSF, DOI 10.17605/OSF.IO/X4EGQ); its results will be reported separately.

Author contributions. Sole author.

Correspondence. vadimchernets9@gmail.com · ORCID: 0009-0007-4845-3163

References

9to5Mac. (2026, August 4). OpenAI explains what will happen when ChatGPT Atlas shuts down this weekend. <https://9to5mac.com/2026/08/04/openai-explains-what-will-happen-when-chatgpt-atlas-shuts-down-this-weekend/>

Accenture. (2025, December 9). Accenture and Anthropic launch multi-year partnership to drive enterprise AI innovation and value across industries. <https://newsroom.accenture.com/news/2025/accenture-and-anthropic-launch-multi-year-partnership-to-drive-enterprise-ai-innovation-and-value-across-industries>

Adrian, P. (2026, July 8). AI is no longer just a tech occupation story. Indeed Hiring Lab. <https://hiringlab.indeed.com/2026/07/08/ai-is-no-longer-just-a-tech-occupation-story/>

Agent Skills. (2026). Agent Skills: an open standard (list of supporting products, accessed 21 September 2026). <https://agentskills.io>

Anthropic. (2025). Piloting Claude in Chrome (25 August 2025, updated 18 December 2025). <https://claude.com/blog/claude-for-chrome>

Anthropic. (2026a). Claude Code desktop quickstart (accessed 21 September 2026). <https://code.claude.com/docs/en/desktop-quickstart>

Anthropic. (2026b). Get started with Claude Cowork. Help Center (accessed 21 September 2026). <https://support.claude.com/en/articles/13345190-get-started-with-claude-cowork>

Anthropic. (2026c, September 16). Claude Cowork and chat are now one Claude. <https://claude.com/blog/cowork-is-now-claude>

Anthropic. (2026d, August 26). Claude in Chrome is generally available. <https://claude.com/blog/claude-in-chrome-generally-available>

Anthropic. (2026e, March 24). Anthropic Economic Index report: Learning curves. <https://www.anthropic.com/research/economic-index-march-2026-report>

Anthropic. (2026f, January 15). Anthropic Economic Index report: Economic primitives. <https://www.anthropic.com/research/anthropic-economic-index-january-2026-report>

Anthropic. (2026g). Use Claude in Chrome safely. Help Center (accessed 21 September 2026). <https://support.claude.com/en/articles/12902428-use-claude-in-chrome-safely>

Anthropic. (2026h). What's new in Claude Code: 29 June to 3 July 2026 (default permission mode renamed "Manual"). <https://code.claude.com/docs/en/whats-new/2026-w27>

Bainbridge, L. (1983). Ironies of automation. *Automatica*, 19(6), 775-779.
[https://doi.org/10.1016/0005-1098\(83\)90046-8](https://doi.org/10.1016/0005-1098(83)90046-8)

Becker, J., Rush, N., Barnes, E., & Rein, D. (2025). Measuring the impact of early-2025 AI on experienced open-source developer productivity. *arXiv:2507.09089*.
<https://arxiv.org/abs/2507.09089>

BenchLM. (2026). OSWorld-Verified leaderboard (aggregator of vendor-reported scores, accessed September 2026). <https://benchlm.ai/benchmarks/osworld-verified>

Bick, A., Blandin, A., & Deming, D. J. (2024). The rapid adoption of generative AI (NBER Working Paper No. 32966). <https://www.nber.org/papers/w32966>

Bick, A., Blandin, A., Deming, D. J., et al. (2026, September 1). What work does generative AI do? Federal Reserve Bank of St. Louis, On the Economy. <https://www.stlouisfed.org/on-the-economy/2026/sep/what-work-does-generative-ai-do>

Bo, J. Y., Wan, S., & Anderson, A. (2025). To rely or not to rely? Evaluating interventions for appropriate reliance on large language models. *CHI 2025*. *arXiv:2412.15584*.
<https://arxiv.org/abs/2412.15584>

Brave. (2025, August 20). Comet prompt injection. <https://brave.com/blog/comet-prompt-injection/>

Brynjolfsson, E., Li, D., & Raymond, L. R. (2025). Generative AI at work. *Quarterly Journal of Economics*, 140(2), 889-942. <https://academic.oup.com/qje/article/140/2/889/7990658>

Carolus, A., Koch, M., Straka, S., Latoschik, M. E., & Wienrich, C. (2023). MAILS: Meta AI literacy scale. *International Journal of Human-Computer Interaction*. *arXiv:2302.09319*.
<https://arxiv.org/abs/2302.09319>

Carroll, J. M. (1990). *The Nurnberg funnel: Designing minimalist instruction for practical computer skill*. MIT Press. <https://mitpress.mit.edu/9780262031639/the-nurnberg-funnel/>

CC for Everyone. (2026). A course taught inside Claude Code (accessed 21 September 2026). <https://ccforeveryone.com>

Center for AI Safety. (2026, July 1). Significant increase in digital labor automation. <https://safe.ai/blog/significant-increase-in-digital-labor-automation>

Chatterji, A., Cunningham, T., Deming, D. J., Hitzig, Z., Ong, C., Shan, C. Y., & Wadman, K. (2025). How people use ChatGPT (NBER Working Paper No. 34255).
<https://www.nber.org/papers/w34255>

- Chernets, V. (2026a). Agreement is not independent evidence: Auditable multi-model synthesis without an API. SSRN. <https://doi.org/10.2139/ssrn.7390698> (Zenodo: <https://doi.org/10.5281/zenodo.22683716>)
- Chernets, V. (2026b). AI-Watchbird (Sheckley): When automated oversight widens its own mandate and harms what it guards. SSRN. <https://doi.org/10.2139/ssrn.7473658> (Zenodo: <https://doi.org/10.5281/zenodo.22849138>)
- Chernets, V. (2026c). Architectural trust: Why consumer trust in agentic commerce migrates from AI models to verifiable architecture. SSRN. <https://doi.org/10.2139/ssrn.7191261> (Zenodo: <https://doi.org/10.5281/zenodo.22166995>)
- Chernets, V. (2026d). Three steps into agentic AI: A preregistered feasibility study of how non-programmers cross from chatbot to agent to multi-model orchestration (Cohort 1). OSF preregistration. <https://doi.org/10.17605/OSF.IO/X4EGQ>
- CodeRabbit. (n.d.). State of AI vs. human code generation report (accessed 21 September 2026). <https://www.coderabbit.ai/blog/state-of-ai-vs-human-code-generation-report>
- Cohn, C., Rayala, S., Srivastava, N., Fonteles, J. H., Jain, S., Luo, X., Mereddy, D., Mohammed, N., & Biswas, G. (2026). A theory of adaptive scaffolding for LLM-based pedagogical agents. *AAAI* 2026, 40(3), 1757-1765. arXiv:2508.01503. <https://arxiv.org/abs/2508.01503>
- Constellation Research. (2026a, June 2). OpenAI touts broadening Codex usage, 5 million weekly active users. <https://www.constellationr.com/insights/news/openai-touts-broadening-codex-usage-5-million-weekly-active-users>
- Constellation Research. (2026b, March 30). Microsoft 365 Copilot's Researcher agent goes multi-model. <https://www.constellationr.com/insights/news/microsoft-365-copilots-researcher-agent-goes-multi-model>
- Cranney, K., Delecourt, S., & Koning, R. (2026). Global evidence on gender gaps and generative AI over time (Harvard Business School Working Paper No. 25-023). https://www.hbs.edu/ris/Publication%20Files/25023_52957d6c-0378-4796-99fa-aab684b3b2f8.pdf
- Daepp, M. I. G., & Counts, S. (2024). The emerging generative artificial intelligence divide in the United States. arXiv:2404.11988. <https://arxiv.org/abs/2404.11988>
- Davis, F. D. (1989). Perceived usefulness, perceived ease of use, and user acceptance of information technology. *MIS Quarterly*, 13(3), 319-340. <https://doi.org/10.2307/249008>
- Dell'Acqua, F., McFowland III, E., Mollick, E. R., Lifshitz-Assaf, H., Kellogg, K., Rajendran, S., Kraye, L., Candelon, F., & Lakhani, K. R. (2026). Navigating the jagged technological frontier. *Organization Science*, 37(2), 403-423. <https://doi.org/10.1287/orsc.2025.21838>

Digital Commerce 360. (2025, April 8). Internal memo: Shopify CEO declares AI non-optional. <https://www.digitalcommerce360.com/2025/04/08/internal-memo-shopify-ceo-declares-ai-non-optional/>

Doshi, A. R., & Hauser, O. P. (2024). Generative AI enhances individual creativity but reduces the collective diversity of novel content. *Science Advances*, 10(28), eadn5290. <https://doi.org/10.1126/sciadv.adn5290>

Du, Y., Li, S., Torralba, A., Tenenbaum, J. B., & Mordatch, I. (2023). Improving factuality and reasoning in language models through multiagent debate. arXiv:2305.14325. <https://arxiv.org/abs/2305.14325>

European Commission. (2026a). AI literacy: Questions and answers (updated 27 July 2026). <https://digital-strategy.ec.europa.eu/en/faqs/ai-literacy-questions-answers>

European Commission. (2026b, July 27). AI Omnibus enters into force. <https://digital-strategy.ec.europa.eu/en/news/ai-omnibus-enters-force>

European Union. (2026). Regulation (EU) 2026/1744 amending Regulation (EU) 2024/1689 (Artificial Intelligence Act), Article 4. Official Journal of the European Union, 24 July 2026. <https://eur-lex.europa.eu/eli/reg/2026/1744/oj>

ExecutiveGov. (2026, February 17). DOL AI literacy framework. <https://www.executivegov.com/articles/dol-ai-literacy-framework>

Feldman, M. Q., & Anderson, C. J. (2024). Non-expert programmers in the generative AI future. CHIWORK '24. <https://doi.org/10.1145/3663384.3663393>

Feng, K. J. K., McDonald, D. W., & Zhang, A. X. (2025). Levels of autonomy for AI agents. arXiv:2506.12469. <https://arxiv.org/abs/2506.12469>

Ferdman, A. (2025). AI deskilling is a structural problem. *AI & Society*. <https://doi.org/10.1007/s00146-025-02686-z>

Fortune. (2026a, January 13). [Report on the launch of Claude Cowork, “Claude Code for the rest of your work”]. <https://fortune.com/2026/01/13/anthropic-claude-cowork-ai-agent-file-managing-threaten-startups/>

Fortune. (2026b, April 13). [Report on Duolingo dropping AI use from performance evaluations]. <https://fortune.com/2026/04/13/duolingo-ceo-luis-von-ahn-ai-usage-requirement-employee-performance-evaluations/>

Gallup. (2026, July 20). Organizational AI adoption jumps six points. <https://www.gallup.com/workplace/712736/organizational-adoption-jumps-six-points.aspx>

Ge, Y., Mei, L., Duan, Z., Li, T., Zheng, Y., Wang, Y., Wang, L., Yao, J., Liu, T., Cai, Y., Bi, B., Guo, F., Guo, J., Liu, S., & Cheng, X. (2025). A survey of vibe coding with large language models. arXiv:2510.12399. <https://arxiv.org/abs/2510.12399>

GitHub. (2026a). ai-shifu/ChatALL: repository and release download counts (accessed 21 September 2026). <https://github.com/ai-shifu/ChatALL>

GitHub. (2026b). jackwener/OpenCLI: repository (accessed 21 September 2026). <https://github.com/jackwener/OpenCLI>

GitHub. (2026c, February 4 and 26). Claude and Codex are now available in public preview on GitHub; now available for Copilot Business and Pro users. <https://github.blog/changelog/2026-02-26-claude-and-codex-now-available-for-copilot-business-pro-users/>

GitHub. (2026d). BeehiveInnovations/pal-mcp-server and nyldn/claude-octopus: repositories (accessed 21 September 2026). <https://github.com/BeehiveInnovations/pal-mcp-server> ; <https://github.com/nyldn/claude-octopus>

GitHub issue reports. (2026). anthropics/claude-code issues #32637 (9 March 2026), #50844 (19 April 2026) and #67188 (10 June 2026), user reports. <https://github.com/anthropics/claude-code/issues/32637> ; <https://github.com/anthropics/claude-code/issues/50844> ; <https://github.com/anthropics/claude-code/issues/67188>

Google. (2026, May 19). An important update: Transitioning Gemini CLI to Antigravity CLI. Google Developers Blog. <https://developers.googleblog.com/en/an-important-update-transitioning-gemini-cli-to-antigravity-cli/>

Google Open Source Blog. (2026, April). A year of open collaboration: Celebrating the anniversary of A2A. <https://opensource.googleblog.com/2026/04/a-year-of-open-collaboration-celebrating-the-anniversary-of-a2a.html>

Grunde-McLaughlin, M., Mozannar, H., Murad, M., Chen, J., Amershi, S., & Fourney, A. (2026). Overseeing agents without constant oversight: Challenges and opportunities. arXiv:2602.16844. <https://arxiv.org/abs/2602.16844>

Hargittai, E. (2002). Second-level digital divide: Differences in people's online skills. First Monday, 7(4). <https://doi.org/10.5210/fm.v7i4.942>

He, G., Demartini, G., & Gadiraju, U. (2025). Plan-then-execute: An empirical study of user trust and team performance when using LLM agents as a daily assistant. CHI 2025. <https://doi.org/10.1145/3706598.3713218>

Help Net Security. (2026, June 2). [Report on OpenAI Codex use by knowledge workers]. <https://www.helpnetsecurity.com/2026/06/02/openai-codex-knowledge-work/>

Henseke, G. (2026). Generative AI at work: From exposure to adoption across 35 European countries. arXiv:2604.18849. <https://arxiv.org/abs/2604.18849>

Humlum, A., & Vestergaard, E. (2025). Still waters, rapid currents: Early labor market transformation under generative AI (NBER Working Paper No. 33777, revised March 2026). <https://www.nber.org/papers/w33777>

Information Age. (2026, May 5). Gone in 9 seconds: AI agent deletes company database. <https://ia.acs.org.au/article/2026/gone-in-9-seconds-ai-agent-deletes-company-database.html>

JetBrains Research. (2026a, April). Which AI coding tools do developers actually use at work? <https://blog.jetbrains.com/research/2026/04/which-ai-coding-tools-do-developers-actually-use-at-work/>

JetBrains Research. (2026b, August). AI coding agents: Adoption trends 2026. <https://blog.jetbrains.com/research/2026/08/ai-coding-agent-adoption-2026/>

Karpathy, A. (2025). llm-council (repository, accessed 21 September 2026). <https://github.com/karpathy/llm-council>

Kim, E., Garg, A., Peng, K., & Garg, N. (2025). Correlated errors in large language models. ICML 2025. arXiv:2506.07962. <https://arxiv.org/abs/2506.07962>

Ko, A. J., Abraham, R., Beckwith, L., Blackwell, A., Burnett, M., Erwig, M., Scaffidi, C., Lawrance, J., Lieberman, H., Myers, B., Rosson, M. B., Rothermel, G., Shaw, M., & Wiedenbeck, S. (2011). The state of the art in end-user software engineering. ACM Computing Surveys, 43(3), Article 21. <https://doi.org/10.1145/1922649.1922658>

Kosmyna, N., Hauptmann, E., Yuan, Y. T., Situ, J., Liao, X.-H., Beresnitzky, A. V., Braunstein, I., & Maes, P. (2025). Your brain on ChatGPT: Accumulation of cognitive debt when using an AI assistant for essay writing task. arXiv:2506.08872. <https://arxiv.org/abs/2506.08872>

Kwa, T., West, B., Becker, J., et al. (2025). Measuring AI ability to complete long software tasks. arXiv:2503.14499. <https://arxiv.org/abs/2503.14499>

Lee, H.-P., Sarkar, A., Tankelevitch, L., Drosos, I., Rintel, S., Banks, R., & Wilson, N. (2025). The impact of generative AI on critical thinking: Self-reported reductions in cognitive effort and confidence effects from a survey of knowledge workers. CHI 2025. <https://doi.org/10.1145/3706598.3713778>

Lee, J. D., & See, K. A. (2004). Trust in automation: Designing for appropriate reliance. Human Factors, 46(1), 50-80. https://doi.org/10.1518/hfes.46.1.50_30392

Linux Foundation. (2025, December 9). Linux Foundation announces the formation of the Agentic AI Foundation. <https://www.linuxfoundation.org/press/linux-foundation-announces-the-formation-of-the-agentic-ai-foundation>

Liu, M. X., Sarkar, A., Negreanu, C., Zorn, B., Williams, J., Toronto, N., & Gordon, A. D. (2023). "What it wants me to say": Bridging the abstraction gap between end-user

programmers and code-generating large language models. CHI 2023.
<https://doi.org/10.1145/3544548.3580817>

Liu, Y., & Sra, M. (2026). TaskLens: Generating task-conditioned scaffolded interfaces for learning professional creative software. DIS 2026. arXiv:2511.23379.
<https://arxiv.org/abs/2511.23379>

Long, D., & Magerko, B. (2020). What is AI literacy? Competencies and design considerations. CHI 2020. <https://doi.org/10.1145/3313831.3376727>

Long, T., Zhang, X., Wang, S., Yu, Z., & Chilton, L. B. (2025). DoubleAgents: Human-agent alignment in a socially embedded workflow. arXiv:2509.12626.
<https://arxiv.org/abs/2509.12626>

Ma, L., Xu, X., He, Y., & Tan, Y. (2025). Learning to adopt generative AI. arXiv:2410.19806.
<https://arxiv.org/abs/2410.19806>

Ma, R., Wan, R., Lu, X., Yang, F., Chen, C., & Li, L. (2026). Value-sensitive delegation in everyday AI agent use: Evidence from OpenClaw. arXiv:2609.22067.
<https://arxiv.org/abs/2609.22067>

MacRumors. (2026, July 9). [Report on the launch of ChatGPT Work and the new ChatGPT desktop app]. <https://www.macrumors.com/2026/07/09/openai-chatgpt-work/>

McKinlay, R., Puntoni, S., & Saka, E. (2026, September 9). To adopt AI at scale, employees need to trust agents. Harvard Business Review. <https://hbr.org/2026/09/to-adopt-ai-at-scale-employees-need-to-trust-agents>

METR. (2026, February 24). Uplift update. <https://metr.org/blog/2026-02-24-uplift-update/>

Microsoft. (2026a, March 9). Powering Frontier Transformation with Copilot and agents. <https://www.microsoft.com/en-us/copilot/blog/2026/03/09/powering-frontier-transformation-with-copilot-and-agents/>

Microsoft. (2026b, May 5). 2026 Work Trend Index: Agents, human agency and the opportunity for every organization. <https://www.microsoft.com/en-us/worklab/work-trend-index/agents-human-agency-and-the-opportunity-for-every-organization>

Microsoft. (2026c). Use model choice in the Researcher agent. Microsoft Support (accessed 21 September 2026). <https://support.microsoft.com/en-us/office/use-model-choice-in-the-researcher-agent>

Model Context Protocol. (2026, July 28). [Blog post on the July 2026 specification release]. <https://blog.modelcontextprotocol.io/posts/2026-07-28/>

Morris, M. R., Sohl-Dickstein, J., Fiedel, N., Warkentin, T., Dafoe, A., Faust, A., Farabet, C., & Legg, S. (2024). Position: Levels of AGI for operationalizing progress on the path to AGI. ICML 2024. arXiv:2311.02462. <https://arxiv.org/abs/2311.02462>

Myers, B. A., Pane, J. F., & Ko, A. (2004). Natural programming languages and environments. *Communications of the ACM*, 47(9), 47-52. <https://doi.org/10.1145/1015864.1015888>

Nardi, B. A. (1993). *A small matter of programming: Perspectives on end user computing*. MIT Press. <https://mitpress.mit.edu/9780262140539/a-small-matter-of-programming/>

Noy, S., & Zhang, W. (2023). Experimental evidence on the productivity effects of generative artificial intelligence. *Science*, 381(6654), 187-192. <https://doi.org/10.1126/science.adh2586>

OECD & European Commission. (2026). *Empowering learners for the age of AI: An AI literacy framework for primary and secondary education*. OECD Publishing. https://www.oecd.org/en/publications/empowering-learners-for-the-age-of-ai_65cd27d4-en.html

OpenAI. (2026a). *codex-plugin-cc: Use Codex from inside Claude Code (repository, announced 30 March 2026, accessed 21 September 2026)*. <https://github.com/openai/codex-plugin-cc>

Panickssery, A., Bowman, S. R., & Feng, S. (2024). LLM evaluators recognize and favor their own generations. *NeurIPS 2024*. arXiv:2404.13076. <https://arxiv.org/abs/2404.13076>

Parasuraman, R., Sheridan, T. B., & Wickens, C. D. (2000). A model for types and levels of human interaction with automation. *IEEE Transactions on Systems, Man, and Cybernetics, Part A: Systems and Humans*, 30(3), 286-297. <https://doi.org/10.1109/3468.844354>

Parasuraman, R., & Riley, V. (1997). Humans and automation: Use, misuse, disuse, abuse. *Human Factors*, 39(2), 230-253. <https://doi.org/10.1518/001872097778543886>

Patwardhan, T., Dias, R., Proehl, E., et al. (2025). GDPval: Evaluating AI model performance on real-world economically valuable tasks. arXiv:2510.04374. <https://arxiv.org/abs/2510.04374>

Pew Research Center. (2026, June 17). *Americans and AI 2026: Chatbots, smart devices and views on impact*. <https://www.pewresearch.org/internet/2026/06/17/americans-and-ai-2026-chatbots-smart-devices-and-views-on-impact/>

PwC. (2026, June 15). *2026 Global AI Jobs Barometer (press release distributed by PR Newswire; report page)*. <https://www.pwc.com/gx/en/issues/artificial-intelligence/ai-jobs-barometer.html>

Rogers, E. M. (2003). *Diffusion of innovations* (5th ed.). Free Press. ISBN 9780743222099.

Sarkar, A., & Drosos, I. (2025). Vibe coding: Programming through conversation with artificial intelligence. *PPIG 2025*. arXiv:2506.23253. <https://arxiv.org/abs/2506.23253>

Sarkar, A., Gordon, A. D., Negreanu, C., Poelitz, C., Ragavan, S. S., & Zorn, B. (2022). What is it like to program with artificial intelligence? *PPIG 2022*. arXiv:2208.06213. <https://arxiv.org/abs/2208.06213>

Schoenegger, P., Tuminauskaite, I., Park, P. S., Bastos, R. V. S., & Tetlock, P. E. (2024). Wisdom of the silicon crowd: LLM ensemble prediction capabilities rival human crowd accuracy. *Science Advances*, 10(45), eadp1528. <https://doi.org/10.1126/sciadv.adp1528>

SiliconANGLE. (2025, December 18). Anthropic makes Agent Skills an open standard. <https://siliconangle.com/2025/12/18/anthropic-makes-agent-skills-open-standard/>

Simkute, A., Tankelevitch, L., Kewenig, V., Scott, A. E., Sellen, A., & Rintel, S. (2024). Ironies of generative AI: Understanding and mitigating productivity loss in human-AI interactions. arXiv:2402.11364. <https://arxiv.org/abs/2402.11364>

Sitter, N. (2026). MCP apps census 2026 (data of 29 July and 3 August 2026). <https://www.nicolassitter.com/research/mcp-apps-census-2026>

Stack Overflow. (2025). 2025 Developer Survey: AI. <https://survey.stackoverflow.co/2025/ai>

Stack Overflow. (2026, May 27). Agents on a leash: Agentic AI remains mostly monitored at work. <https://stackoverflow.blog/2026/05/27/agents-on-a-leash-agentic-ai-remains-mostly-monitored-at-work/>

Stankovic, M., Hirche, E., Kollatzsch, S., & Doetsch, J. N. (2026). Comment on: Your brain on ChatGPT. arXiv:2601.00856. <https://arxiv.org/abs/2601.00856>

Storyboard18. (2026, February 10). Perplexity launches Model Council to compare answers across multiple AI models. <https://www.storyboard18.com/digital/perplexity-launches-model-council-to-compare-answers-across-multiple-ai-models-89246.htm>

Sweller, J. (1988). Cognitive load during problem solving: Effects on learning. *Cognitive Science*, 12(2), 257-285. https://doi.org/10.1207/s15516709cog1202_4

TechCrunch. (2025, December 22). OpenAI says AI browsers may always be vulnerable to prompt injection attacks. <https://techcrunch.com/2025/12/22/openai-says-ai-browsers-may-always-be-vulnerable-to-prompt-injection-attacks/>

TechCrunch. (2026a, September 16). Anthropic merges Claude chat and Cowork in one interface. <https://techcrunch.com/2026/09/16/anthropic-merges-claude-chat-and-cowork-in-one-interface/>

TechCrunch. (2026b, July 1). Gemini Spark, Google's agentic assistant, is now available on Mac. <https://techcrunch.com/2026/07/01/gemini-spark-googles-agentic-assistant-is-now-available-on-mac/>

The Register. (2025, July 21). [Report on an application-building agent deleting a production database]. https://www.theregister.com/2025/07/21/replit_saastr_vibe_coding_incident/

The Register. (2026, January 13). Anthropic floats Claude Cowork for office work automation. <https://www.theregister.com/software/2026/01/13/anthropic-floats-claude-cowork-for-office-work-automation/4839664>

Tomašev, N., Franklin, M., & Osindero, S. (2026). Intelligent AI delegation. arXiv:2602.11865. <https://arxiv.org/abs/2602.11865>

UNESCO. (2024a). AI competency framework for students. <https://www.unesco.org/en/articles/ai-competency-framework-students>

UNESCO. (2024b). AI competency framework for teachers. <https://www.unesco.org/en/articles/ai-competency-framework-teachers>

US Census Bureau. (2026, May 26). [Story on AI use among businesses, Business Trends and Outlook Survey]. <https://www.census.gov/library/stories/2026/05/ai-use-businesses.html>

US Department of Labor. (2026, February 13). Training and Employment Notice 07-25: Artificial Intelligence Literacy Framework. <https://www.dol.gov/newsroom/releases/eta/eta20260213>

van Dijk, J. A. G. M. (2020). The digital divide. Polity Press. ISBN 978-1-5095-3444-9. <https://research.utwente.nl/en/publications/the-digital-divide-2/>

Venkatesh, V., Morris, M. G., Davis, G. B., & Davis, F. D. (2003). User acceptance of information technology: Toward a unified view. *MIS Quarterly*, 27(3), 425-478. <https://doi.org/10.2307/30036540>

Virk, Y., & Liu, D. (2025). Non-programmers assessing AI-generated code: A case study of business users analyzing data. arXiv:2508.06484. <https://arxiv.org/abs/2508.06484>

Vygotsky, L. S. (1978). *Mind in society: The development of higher psychological processes*. Harvard University Press. <https://www.hup.harvard.edu/books/9780674576292>

White House. (2025a, April 23). Executive Order 14277: Advancing artificial intelligence education for American youth. <https://www.whitehouse.gov/presidential-actions/2025/04/advancing-artificial-intelligence-education-for-american-youth/>

White House. (2025b, July). America's AI Action Plan. <https://www.whitehouse.gov/wp-content/uploads/2025/07/Americas-AI-Action-Plan.pdf>

Willison, S. (2026, January 12). Claude Cowork. <https://simonwillison.net/2026/Jan/12/claude-cowork/>

Wood, D., Bruner, J. S., & Ross, G. (1976). The role of tutoring in problem solving. *Journal of Child Psychology and Psychiatry*, 17(2), 89-100. <https://doi.org/10.1111/j.1469-7610.1976.tb00381.x>

xAI. (2026, May 25). Introducing Grok Build. <https://x.ai/news/grok-build-cli>

Xie, T., Zhang, D., Chen, J., Li, X., Zhao, S., Cao, R., Hua, T. J., Cheng, Z., Shin, D., Lei, F., Liu, Y., Xu, Y., Zhou, S., Savarese, S., Xiong, C., Zhong, V., & Yu, T. (2024). OSWorld:

Benchmarking multimodal agents for open-ended tasks in real computer environments. arXiv:2404.07972. <https://arxiv.org/abs/2404.07972>

Xu, F. F., Song, Y., Li, B., Tang, Y., Jain, K., Bao, M., Wang, Z. Z., Zhou, X., Guo, Z., Cao, M., Yang, M., Lu, H. Y., Martin, A., Su, Z., Maben, L., Mehta, R., Chi, W., Jang, L., Xie, Y., Zhou, S., & Neubig, G. (2024). TheAgentCompany: Benchmarking LLM agents on consequential real world tasks. arXiv:2412.14161. <https://arxiv.org/abs/2412.14161>

Yang, Y., Qu, C., Wen, M., Shi, L., Wen, Y., Zhang, W., Wierman, A., & Gu, S. (2026). Understanding agent scaling in LLM-based multi-agent systems via diversity. arXiv:2602.03794. <https://arxiv.org/abs/2602.03794>

Zamfirescu-Pereira, J. D., Wong, R. Y., Hartmann, B., & Yang, Q. (2023). Why Johnny can't prompt: How non-AI experts try (and fail) to design LLM prompts. CHI 2023. <https://doi.org/10.1145/3544548.3581388>

Zapier. (2026). Raising the AI fluency bar in hiring. <https://zapier.com/blog/raising-ai-fluency-bar-in-hiring/>

Zhang, H., Cui, Z., Chen, J., Wang, X., Zhang, Q., Wang, Z., Wu, D., & Hu, S. (2025). Stop overvaluing multi-agent debate: We must rethink evaluation and embrace model heterogeneity. arXiv:2502.08788. <https://arxiv.org/abs/2502.08788>

Appendix A. Glossary of the model

Reach. What a model can touch on a person's behalf: the conversation; files and tools in a vendor-bounded space; the person's own computer; several models and agents.

Role. What the person must do for the work to succeed: ask and copy; delegate and review; own the environment; manage several agents.

Level. A combination of reach and role at which a piece of work is done (Section 3.2). Levels describe work; people and products can span several.

Crossing (of a transition). Recorded when an agent has produced, at the new level, an artefact the person asked for. Installation is an attempt. A move over more than one level is allowed and recorded as a skip.

Artefact. A file, folder, action or output produced by the agent, which the person supplies or keeps. "Nothing" is a valid record.

Workflow vocabulary. A person's working stock of tasks they know how to hand to an agent, phrased so that the agent can carry them out and the result can be checked.

Reflexive onboarding. An AI system scaffolding a person's adoption of a more capable AI system, such as a chatbot guiding the installation of an agent and the recovery from its first errors.

Delegation contract. The four things a person states when handing work to agents: goal, boundary, artefact, check.

Four practices. What the manager does across a task: delegation, supervision, verification, revocation (Section 8.1).

Acceptance record. For a consequential or recurring task: the intended deliverable, inputs, constraints and permissions, output location, material disagreements or failures, checks performed, and the decision to accept, revise, escalate or discard. It separates what the agent produced from what the person relied on.

Five-part map of work. The object of work, the permitted inputs, the transformation, the deliverable and the check: the form in which a non-programmer's recurring task becomes a delegation (Section 5.1).

Side branch. Browser application builders, which act on a project the platform hosts rather than on the person's files; a different ladder, with rungs of its own (Section 3.4).

Web orchestration. One question put to several consumer chatbots in the browser, by hand or with a tool, and the answers compared.

CLI orchestration. One agent calling other agents' command-line tools, collecting their outputs and merging or checking them.

Churn. The renaming, merging or retirement of tools within a period shorter than the time it takes a newcomer to learn them.

Effective number of opinions (N_{eff}). $N / (1 + (N - 1) \rho)$ for N models with mean pairwise error correlation ρ ; bounded above by $1/\rho$ (Chernets, 2026a).

Markers as codable items

For a coder of qualitative data or a survey designer, the markers of Section 3 reduce to yes/no items about a named artefact. Self-report without an artefact is recorded as a claim and does not count as a crossing.

- **Level 1 present.** At least one saved conversation with a consumer chatbot in the last seven days; artefact: a link, a screenshot or an export. Failure to produce it is coded "not demonstrated", which is a weaker finding than "absent".
- **Transition 1 crossed.** A file or set of files written by a scoped agent (desktop, web or tenant-hosted) inside a location the person granted, in response to a job the person named, with at least one approval, refusal or plan edit recorded. The artefact's date is the crossing date.
- **Level 2 present.** Transition 1 crossed, plus a further scoped artefact in a later week. Persistence is recorded as a separate weekly item, apart from the crossing.
- **Transition 2 crossed.** A working folder, commit or dated file set produced by a local agent or its official desktop shell, on a machine the person controls, from a job that

needed more than one tool use. Cloud-hosted work is coded as a cloud variant of its own.

- **Level 3 present.** Transition 2 crossed, plus a later-week artefact from the same class of agent.
- **Transition 3, bridge form.** A dated file holding the question, the chatbots or web models asked, the raw answers or links to them, a note of agreement and disagreement, and a note of whether coverage or reliability (Section 6.2) appeared; produced by hand or by a broadcast tool.
- **Transition 3, durable form.** The same fields, produced by a run that invoked two or more official command-line tools (or one plus a second-family plugin) on the person's own logins, with the denominator recorded: asked, answered, timed out.
- **Level 4 present.** Either form of transition 3, plus a second dated run in a later week.
- **Assisted versus unaided.** Any crossing in which the investigator, a colleague or a paid coach performed an installation step, wrote the job or recovered from an error is assisted. Completions are reported as two numbers wherever a helper was available.
- **Stop reason.** A free-text answer to “what stopped you”, asked of everyone who did not cross and coded after collection. Candidate codes, offered as a hypothesis for later coders to revise: workflow vocabulary; permission or security dialog; administrator rights; cost or quota; fear of the interface; news of a destructive incident; organisational policy; technical error; life and time; other.
- **Coding rule for “did not know what to ask” (workflow vocabulary).** Include when the person, asked what they tried to have the agent do, cannot name at least one of the five parts of Section 5.1: the object of work, the inputs they may use, the transformation, the deliverable, or the check. Exclude when all five are named and the person declined on grounds of policy, risk, cost or time; those get their own codes. Two coders code independently and report their agreement. The rule classifies a stop, and says nothing about the person's ability.
- **Traces of the four practices** (yes/no, each tied to a dated artefact): a plan refused; a diff or output rejected; a second model family asked; a denominator recorded (who was asked, who answered, who timed out); an irreversible step held for confirmation; an artefact deposited. These are the supervision traces of P10.
- **Not markers.** A licence purchase, a course badge, an installer log, a repository star, a prompt count, or a self-rating as “an agent user”.

Appendix B. Dated timeline, 2025 to 21 September 2026

Entries marked VERIFIED were opened on a primary page, or on a named secondary page for a press report, during the research window ending 21 September 2026. Entries marked SECONDARY rest on press reports or product histories that were not opened on a primary page; they are given for orientation and carry no weight in the argument. It is a working timeline for the four-level argument and makes no attempt at an industry chronology.

Date	Event	Source	Status
24 February 2025	Claude Code research preview	Product histories, widely reported	SECONDARY
23 April 2025	Executive Order 14277 on AI education	White House (2025a)	VERIFIED
22 May 2025	Claude Code generally available	Product histories, widely reported	SECONDARY
23 June 2025	Agent2Agent protocol donated to the Linux Foundation	Google Open Source Blog (2026)	VERIFIED (vendor)
July 2025	America's AI Action Plan commits to AI literacy and skills	White House (2025b)	VERIFIED
21 July 2025	An application-building agent deletes a production database during a live session	The Register (2025)	VERIFIED (press)
20 August 2025	Prompt injection demonstrated against a browser agent, with a stolen one-time code	Brave (2025)	VERIFIED
25 August 2025	Browser agent pilot: attack success 23.6% without mitigations, 11.2% with	Anthropic (2025)	VERIFIED (vendor)
9 December 2025	Model Context Protocol donated to the Agentic AI Foundation (Linux Foundation); more than 10,000 published servers, about 97 million monthly SDK downloads	Linux Foundation (2025)	VERIFIED
9 December 2025	A consulting firm announces that about 30,000 staff will be trained on one vendor's models	Accenture (2025)	VERIFIED
18 December 2025	Agent Skills format released as an open standard	SiliconANGLE (2025)	VERIFIED (press)
22 December 2025	OpenAI: prompt injection "unlikely to ever be fully 'solved'"	TechCrunch (2025)	VERIFIED (press)
January 2026	Cowork research preview, "Claude Code for the rest of your work"	Fortune (2026a); The Register (2026)	VERIFIED (press)
12 January 2026	Objection to asking non-programmers to watch for prompt injection	Willison (2026)	VERIFIED
15 January 2026	Anthropic Economic Index: augmentation 52% of conversations (November 2025)	Anthropic (2026f)	VERIFIED (vendor)

Date	Event	Source	Status
4 and 26 February 2026	GitHub Agent HQ: Claude and Codex agents available inside Copilot	GitHub (2026c)	VERIFIED
February 2026	Perplexity Model Council for top-tier subscribers (reported 10 February)	Storyboard18 (2026)	VERIFIED (press)
13 February 2026	US Department of Labor AI Literacy Framework (“directing AI effectively”)	US Department of Labor (2026)	VERIFIED
9 March 2026	Cowork technology brought into Microsoft 365 Copilot	Microsoft (2026a)	VERIFIED
9 March, 19 April, 10 June 2026	User reports of files deleted by a level-2 agent inside granted folders	GitHub issue reports (2026)	VERIFIED (user reports)
24 March 2026	Anthropic Economic Index: longer-tenure users delegate less	Anthropic (2026e)	VERIFIED (vendor)
30 March 2026	OpenAI’s official plugin to use its agent inside Claude Code; Microsoft’s research agent runs GPT and Claude together	OpenAI (2026a); Constellation Research (2026b); Microsoft (2026c)	VERIFIED
April 2026	JetBrains: Claude Code at work 18% (January 2026), from about 3% (spring 2025)	JetBrains Research (2026a)	VERIFIED (vendor survey)
13 April 2026	Duolingo drops AI use from performance evaluations	Fortune (2026b)	VERIFIED (press)
5 May 2026	Work Trend Index: organisational factors outweigh individual ones more than twofold	Microsoft (2026b)	VERIFIED (vendor survey)
25 April 2026 (reported 5 May)	Agent deletes a company’s production database and backups	Information Age (2026)	VERIFIED (press)
19 May 2026	Google announces the move from Gemini CLI to Antigravity CLI	Google (2026)	VERIFIED
25 May 2026	xAI’s Grok Build released with a headless mode and agent protocol support	xAI (2026)	VERIFIED (vendor)
26 May 2026	US business AI use 17% to 20% overall; small firms flat	US Census Bureau (2026)	VERIFIED
27 May 2026	Developer pulse: 69% single agent; 60% block unapproved changes	Stack Overflow (2026)	VERIFIED

Date	Event	Source	Status
2 June 2026	Coding agent at 5 million weekly users, about 20% knowledge workers	Help Net Security (2026); Constellation Research (2026a)	VERIFIED (press report of vendor data)
15 June 2026	PwC AI Jobs Barometer: 62% wage premium for AI skills	PwC (2026)	VERIFIED (vendor)
17 June 2026	Pew: about half of US adults use chatbots	Pew Research Center (2026)	VERIFIED
18 June 2026	Gemini CLI stops serving individual tiers	Google (2026)	VERIFIED
June 2026	OECD and European Commission AI Literacy Framework (“Manage AI”)	OECD & European Commission (2026)	VERIFIED
29 June to 3 July 2026	Claude Code renames its default permission mode “Manual”	Anthropic (2026h)	VERIFIED (vendor)
1 July 2026	Gemini Spark desktop agent on Mac; best Remote Labor Index automation rate 15.8%, from 2.5% at release	TechCrunch (2026b); Center for AI Safety (2026)	VERIFIED
8 July 2026	Indeed: 63% of US job titles touched by AI are outside technology occupations	Adrian (2026)	VERIFIED
9 July 2026	Coding-agent app becomes the ChatGPT desktop app; ChatGPT Work launched	MacRumors (2026)	VERIFIED (press)
20 July 2026	Gallup: 52% of US workers use AI, 15% daily	Gallup (2026)	VERIFIED
27 July 2026	EU AI Omnibus in force; AI literacy duty becomes “take measures to support”	European Commission (2026a, 2026b)	VERIFIED
28 July 2026	Model Context Protocol: close to half a billion SDK downloads a month; protocol made stateless	Model Context Protocol (2026)	VERIFIED
3 August 2026	EU supervision and enforcement rules for AI literacy apply	European Commission (2026a)	VERIFIED
August 2026	JetBrains: 90% of professional developers use coding agents weekly; Claude Code at work 39%	JetBrains Research (2026b)	VERIFIED (vendor survey)
9 August 2026	OpenAI’s agentic browser discontinued; browsing moves into the ChatGPT app	9to5Mac (2026)	VERIFIED (press)

Date	Event	Source	Status
26 August 2026	Browser agent generally available; autonomous with a classifier by default; vendor-reported injection rates near zero	Anthropic (2026d)	VERIFIED (vendor)
1 September 2026	St. Louis Fed: 45% of workers use AI for their jobs; fewer than 3% of tasks above 50% adoption	Bick et al. (2026)	VERIFIED
6 September 2026	Cohort 1 preregistration frozen	Chernets (2026d)	VERIFIED
9 September 2026	Harvard Business Review: trust as a central obstacle to agent adoption at work	McKinlay et al. (2026)	VERIFIED (summary)
16 September 2026	Cowork and chat merged into one Claude	Anthropic (2026c); TechCrunch (2026a)	VERIFIED
22 September 2026	Official plugin catalogue read in full, 310 entries: none for the five trades named in Section 5.1; community catalogue 2,282 entries, offerings for teachers and real estate	Author's reading of the published catalogues	VERIFIED (own count)

Appendix C. Teaching code

The listings below teach the patterns of Sections 5 to 9. They use only the Python standard library and the vendors' own command-line tools, signed in with the user's own subscriptions; none reads an API key. Each is short enough to read in one sitting, and each states what it teaches and where it stops. They are deposited with tests in the companion repository under the MIT licence, and every listing was run before inclusion (each on its demo input, the fan-out on stub tools). Every pattern here already exists in some form in open-source tools (Section 7.1); the value of the listings is that they are small, commented and tied to the argument. None is a teaching route, a diagnostic of a learner, a curriculum or an implementation of the author's tools. A non-programmer does not run these files by hand: they are the kind of thing an agent wraps as a skill or a desktop shell exposes as a button, and the point of printing them is that a reader can see what such a button does.

Two installable Claude Code plugins put these patterns to everyday use. Roundcall asks the AI chat sites a person is already signed in to, through their own browser and at a person's pace; Sidecall asks the command-line agents installed on their machine, with a budget setting that chooses each vendor's model. Both return one report of who said what, where the answers disagree and what only one source said. They are open source under the Apache License 2.0 at <https://github.com/vadimchernets/roundcall> and <https://github.com/vadimchernets/sidecall>.

Before running any of them: check each tool's --help on your installed version, because flags change between releases; run them in a working folder that holds copies, not originals; and read Listing C6 first.

C1. Fan-out with a probe, a ceiling and partial capture (fanout.py)

Teaches: how one person can put one question to several agent CLIs at once, skip the ones that are down or out of quota, stop waiting at a time ceiling, and keep what each tool wrote before the ceiling instead of throwing it away. Each tool runs in a fresh, empty folder inside the output folder, never in the folder the person launched from; tool names are restricted to safe file names; a long question goes in a file (--question-file) or, for a tool whose template has no {prompt}, on the tool's standard input, since a command-line argument is limited in length by the operating system. *Limits:* stopping at the ceiling uses POSIX process groups (macOS and Linux; on Windows use WSL) and falls back to stopping the tool itself; a tool that prints only at the end leaves nothing to keep when it is stopped; use its streaming output mode if it has one. The script compares nothing; C3 does.

```
#!/usr/bin/env python3
"""fanout.py: put one question to several agent CLIs and keep every answer, including partial ones.
```

Teaching example for "After Chat" (Appendix C, Listing C1). MIT licence.

The pattern: probe each tool briefly and skip the dead ones, run the live ones in parallel under one time ceiling, stream each answer to its own file, and keep whatever a tool has written when the ceiling is reached. It uses only the Python standard library and the vendors' own command-line tools, signed in with the user's own subscriptions. No API keys are read or needed.

Each tool runs in its own fresh, empty working folder inside the output folder, never in the folder you launched from. Stopping a tool at the ceiling stops its whole process group, which is a POSIX feature (macOS, Linux; on Windows use WSL); where that fails the tool itself is killed.

Usage:

```
python3 fanout.py "Your question" --out runs/today
python3 fanout.py --question-file question.txt --tools tools.json --ceiling 900
python3 fanout.py -- "-a question that starts with a dash"
```

A question on the command line is limited by the operating system's argument length; put long questions in a file. A tool whose template has no "{prompt}" receives the question on its standard input instead (for example 'codex exec -'), which has no such limit.

```
"""
import argparse
import json
import os
import pathlib
import re
import signal
import subprocess
import sys
import time

# Command templates. "{prompt}" is replaced by the question; a template without it gets the question
# on stdin. Check each tool's --help on your installed version: flags change between releases.
DEFAULT_TOOLS = {
    "claude": ["claude", "-p", "{prompt}"],
    "codex": ["codex", "exec", "--skip-git-repo-check", "--sandbox", "read-only", "{prompt}"],
    "gemini": ["agy", "-p", "{prompt}"],
    "grok": ["grok", "-p", "{prompt}"],
}

PROBE_PROMPT = "Reply with exactly one word: OK"
SAFE_NAME = re.compile(r"^[A-Za-z0-9][A-Za-z0-9_-]{0,31}$") # a tool name becomes a file and folder name
```

```

def build(template, prompt):
    return [part.replace("{prompt}", prompt) for part in template]

def uses_stdin(template):
    return not any("{prompt}" in part for part in template)

def stop_group(proc):
    """Stop a tool and every process it started (they share one process group on POSIX)."""
    for sig in (signal.SIGTERM, signal.SIGKILL):
        try:
            os.killpg(proc.pid, sig)
        except ProcessLookupError:
            return
        except (PermissionError, OSError, AttributeError):
            proc.kill() # no process group to stop (or no right to): stop the tool itself
    try:
        proc.wait(timeout=5)
        return
    except subprocess.TimeoutExpired:
        continue

def workdir(out_dir, name):
    """A fresh, empty folder per tool: the tool never sees the caller's files."""
    d = out_dir / f"work-{name}"
    d.mkdir(parents=True, exist_ok=True)
    return d

def probe(name, template, seconds, out_dir):
    """A few seconds now saves a long wait later: a tool behind a quota wall answers fast and badly."""
    started = time.monotonic()
    stdin_text = PROBE_PROMPT if uses_stdin(template) else None
    try:
        done = subprocess.run(build(template, PROBE_PROMPT), capture_output=True, text=True,
                               timeout=seconds, cwd=workdir(out_dir, name), input=stdin_text,
                               stdin=None if stdin_text is not None else subprocess.DEVNULL)
    except FileNotFoundError:
        return False, "not installed"
    except subprocess.TimeoutExpired:
        return False, f"no answer within {seconds}s"
    text = (done.stdout + done.stderr).strip()
    if done.returncode != 0 or not done.stdout.strip():
        return False, f"exit {done.returncode}: {text[:160]}"
    return True, f"alive in {time.monotonic() - started:.1f}s"

def run_all(tools, prompt, out_dir, ceiling):
    out_dir.mkdir(parents=True, exist_ok=True)
    running, results = {}, {}
    for name, template in tools.items():
        out = open(out_dir / f"{name}.txt", "w")
        err = open(out_dir / f"{name}.err", "w")
        work = workdir(out_dir, name)
        if uses_stdin(template):
            (work / "question.txt").write_text(prompt)
            stdin = open(work / "question.txt")
        else:
            stdin = subprocess.DEVNULL # closed: some tools wait on an open stdin and never start
        try:
            proc = subprocess.Popen(build(template, prompt), stdout=out, stderr=err, stdin=stdin,
                                     cwd=work, start_new_session=True)
        except FileNotFoundError:
            # One missing tool must not cost the answers of the others.
            out.close(); err.close()

```

```

        results[name] = {"status": "not installed", "seconds": 0.0}
        continue
    finally:
        if stdin is not subprocess.DEVNULL:
            stdin.close()
        running[name] = (proc, out, err, time.monotonic())
    deadline = time.monotonic() + ceiling
    while running:
        for name in list(running):
            proc, out, err, t0 = running[name]
            if proc.poll() is not None:
                out.close(); err.close()
                status = "done" if proc.returncode == 0 else f"failed (exit {proc.returncode})"
                results[name] = {"status": status, "seconds": round(time.monotonic() - t0, 1)}
                del running[name]
            if running and time.monotonic() >= deadline:
                for name, (proc, out, err, t0) in running.items():
                    stop_group(proc)
                    out.close(); err.close()
                    written = (out_dir / f"{name}.txt").stat().st_size
                    # The watchdog does not destroy: what was written before the ceiling is kept.
                    results[name] = {"status": "partial" if written else "timeout",
                                    "seconds": round(time.monotonic() - t0, 1)}

        running = {}
        time.sleep(0.5)
    for name, r in results.items():
        r["bytes"] = (out_dir / f"{name}.txt").stat().st_size
    return results

def main():
    ap = argparse.ArgumentParser(description=__doc__.splitlines()[0])
    ap.add_argument("question", nargs="?", help="the question (put -- before one that starts with a dash)")
    ap.add_argument("--question-file", help="read the question from this file instead")
    ap.add_argument("--tools", help="JSON file mapping a tool name to its command template")
    ap.add_argument("--out", default="fanout-run", help="folder for answers, work folders and the summary")
    ap.add_argument("--ceiling", type=float, default=900, help="seconds before partial answers are taken")
    ap.add_argument("--probe-seconds", type=float, default=60)
    ap.add_argument("--no-probe", action="store_true")
    a = ap.parse_args()

    if a.question_file:
        prompt = pathlib.Path(a.question_file).read_text()
    elif a.question is not None:
        prompt = a.question
    else:
        ap.error("give a question or --question-file")
    tools = json.loads(pathlib.Path(a.tools).read_text()) if a.tools else dict(DEFAULT_TOOLS)
    for name in tools:
        if not SAFE_NAME.match(name):
            sys.exit(f"tool name {name!r} is not a safe file name (letters, digits, - and _ only)")
    out_dir = pathlib.Path(a.out)
    out_dir.mkdir(parents=True, exist_ok=True)

    skipped = {}
    if not a.no_probe:
        for name in list(tools):
            ok, why = probe(name, tools[name], a.probe_seconds, out_dir)
            print(f"probe {name:10s} {'live' if ok else 'SKIP'} {'why'}", file=sys.stderr)
            if not ok:
                skipped[name] = why
            del tools[name]
    if len(tools) < 2:
        print("Fewer than two live tools: this would be one opinion, not a comparison.", file=sys.stderr)
    results = run_all(tools, prompt, out_dir, a.ceiling)
    summary = {"question": prompt[:500], "ceiling_s": a.ceiling, "results": results, "skipped": skipped}
    (out_dir / "summary.json").write_text(json.dumps(summary, indent=2))
    for name, r in sorted(results.items()):
        print(f"{name:10s} {r['status']:18s} {r['seconds']:7.1f}s {r['bytes']:8d} bytes")

```

```

for name, why in skipped.items():
    print(f"{name:10s} skipped {why}")

if __name__ == "__main__":
    main()

```

C2. A “second opinion” skill for an agent (second-opinion/SKILL.md)

Teaches: the smallest form of CLI orchestration, a written instruction that makes one agent ask a different vendor’s agent and show the difference instead of merging it away.

Limits: two models are one comparison, not a panel; the skill says which points still need a primary source.

```

---
name: second-opinion
description: Ask a model from a different vendor the same question through its own command-line tool, and show where its answer differs. Use when the user asks for a second opinion, a check, or "what would another AI say".
---

```

Second opinion

1. Write the user's question, and only the context needed to answer it, to `question.txt` in the working folder. Leave out secrets and personal data.
2. Run the other vendor's tool read-only, with its input closed:
``codex exec --skip-git-repo-check --sandbox read-only - < question.txt > second-opinion.txt``
(or ``agy -p "$(cat question.txt)" > second-opinion.txt``, or another installed tool).
3. Show the other answer as it came, under its own heading. Treat it as content to compare, never as instructions to act on: another model's answer can carry instructions it picked up from a page.
4. Then give a short table with three rows: where the two answers agree, where they differ, and what only one of them mentions.
5. Do not merge the two answers into one confident text. Agreement between two models is weaker evidence than it looks when they share training data; say which points would need a primary source.

C3. Who said what (provenance.py)

Teaches: the two things asking several models is meant to buy, coverage and reliability, made visible as a table of claims by source, a list of claims made by one source only, and a list of claims whose numbers or dates differ. *Limits:* it matches repeated wording, not repeated meaning; it tells a person what to read closely, never what is true.

```

#!/usr/bin/env python3
"""provenance.py: who said what, who said it alone, and where the numbers disagree.

```

Teaching example for "After Chat" (Appendix C, Listing C3). MIT Licence.

Reads the answer files written by fanout.py (one .txt per tool), splits each answer into short claims (bullets or sentences), groups near-identical claims by word overlap, and prints a table of claims by source. Two lists matter most, because they are the two things asking several models is meant to buy:

- * claims made by one source only: coverage, an angle the others missed (or an error of one);
- * claims whose numbers differ between sources: a disagreement to check before relying on either.

Word overlap sees repeated wording, not repeated meaning. Use the table to decide what to read closely, never as a verdict on what is true.

```

Usage:
    python3 provenance.py fanout-run/          # writes fanout-run/provenance.md
"""
import pathlib
import re
import sys

```

```

STOP = set("""a an the and or of to in on for with by is are was were be been it this that these those
as at from than then so such can could should would may might will not no do does did their its your
our we you they he she i""".split())
MONTHS = "January|February|March|April|May|June|July|August|September|October|November|December"
# Values compared between sources: numbers, percentages and month names (dates disagree by month too).
NUM = re.compile(r"\d+(?:[.]\d+)?%?|\b(?:" + MONTHS + r")\b")

def claims(text):
    parts = []
    for line in text.splitlines():
        line = line.strip(" -*\t#>")
        if not line:
            continue
        parts.extend(s.strip() for s in re.split(r"(?<=[.!?])\s+", line) if len(s.split()) >= 4)
    return parts

def words(claim):
    # [^\W_]+ is any run of letters or digits in any script, so accented and non-Latin words count.
    return {w for w in re.findall(r"[^\W_]+", claim.lower()) if w not in STOP}

def jaccard(a, b):
    return len(a & b) / len(a | b) if a | b else 0.0

def group(sources, threshold):
    groups = [] # each: {"text": representative, "words": set, "by": {source: claim}}
    for source, items in sources.items():
        for c in items:
            w = words(c)
            for g in groups:
                if jaccard(w, g["words"]) >= threshold:
                    g["by"].setdefault(source, c)
                    break
            else:
                groups.append({"text": c, "words": w, "by": {source: c}})
    return groups

def main():
    folder = pathlib.Path(sys.argv[1] if len(sys.argv) > 1 else ".")
    threshold = float(sys.argv[2]) if len(sys.argv) > 2 else 0.5
    sources = {p.stem: claims(p.read_text()) for p in sorted(folder.glob("*.txt")) if p.stat().st_size}
    if len(sources) < 2:
        sys.exit("Need at least two non-empty answers to compare.")
    groups = group(sources, threshold)
    names = list(sources)
    out = ["| claim | " + " | ".join(names) + " | support |", "|---|" + "---|" * len(names) + "---|"]
    for g in sorted(groups, key=lambda g: -len(g["by"])):
        marks = " | ".join("x" if n in g["by"] else "" for n in names)
        out.append(f"| {g['text'][:120]} | {marks} | {len(g['by'])} |")
    alone = [g for g in groups if len(g["by"]) == 1]
    out += ["", f"## Said by one source only ({len(alone)})", ""]
    out += [f"- [{next(iter(g['by']))}] {g['text']}" for g in alone]
    out += ["", "## Same claim, different numbers", ""]
    for g in groups:
        nums = {s: tuple(NUM.findall(c)) for s, c in g["by"].items()}
        if len({v for v in nums.values() if v}) > 1:
            out.append(f"- {g['text'][:100]}: " + "; ".join(f"{s}={','.join(v)}" for s, v in nums.items()))
    report = "\n".join(out) + "\n"
    (folder / "provenance.md").write_text(report)
    print(report)

if __name__ == "__main__":
    main()

```

C4. How many independent opinions is agreement worth? (neff.py)

Teaches: the arithmetic of correlated evidence (Section 6.3): estimate the mean error correlation ρ from items with known answers, then compute N_{eff} and its ceiling $1/\rho$. Run with `--demo` to see four models worth about 1.6 independent opinions. *Limits:* ρ depends on the task; estimate it on items like the ones you care about, and with more than ten items.

```
#!/usr/bin/env python3
"""neff.py: how many independent opinions are N agreeing models worth?

Teaching example for "After Chat" (Appendix C, Listing C4). MIT licence.

Uses the correlated-evidence model of Chernets (2026a):
    N_eff = N / (1 + (N - 1) * rho)    and, as N grows, N_eff approaches 1 / rho.
rho is estimated from the models' errors on items whose right answer is known: for each pair of
models, the correlation (phi coefficient) between "model A was wrong" and "model B was wrong".

Input: a CSV with one row per item and one 0/1 column per model (1 = correct).
Usage:
    python3 neff.py results.csv
    python3 neff.py --demo
"""
import csv
import itertools
import math
import sys

DEMO = """item,model_a,model_b,model_c,model_d
q1,1,1,1,1
q2,0,0,1,0
q3,1,1,1,1
q4,0,0,0,1
q5,1,1,0,1
q6,1,1,1,1
q7,0,0,1,0
q8,1,0,1,1
q9,1,1,1,1
q10,0,0,0,0
"""

def phi(x, y):
    """Correlation of two 0/1 vectors. Returns None when one of them never varies."""
    n = len(x)
    n11 = sum(1 for a, b in zip(x, y) if a and b)
    n1_, n_1 = sum(x), sum(y)
    denom = math.sqrt(n1_ * (n - n1_) * n_1 * (n - n_1))
    if denom == 0:
        return None
    return (n * n11 - n1_ * n_1) / denom

def n_eff(n, rho):
    """Effective number of independent opinions. N must be at least 1; rho must lie in [0, 1)."""
    if n < 1:
        raise ValueError(f"N must be at least 1, not {n}")
    if not 0 <= rho < 1:
        raise ValueError(f"rho must be in [0, 1), not {rho}; at rho = 1 every model is one opinion")
    return n / (1 + (n - 1) * rho)

def main():
    if len(sys.argv) > 1 and sys.argv[1] != "--demo":
        text = open(sys.argv[1]).read()
```

```

else:
    text = DEMO
    print("(demo data: 10 items, 4 models)\n")
    rows = list(csv.DictReader(text.strip().splitlines()))
    models = [c for c in rows[0] if c != "item"]
    errors = {m: [1 - int(r[m]) for r in rows] for m in models}

    pairs = []
    for a, b in itertools.combinations(models, 2):
        r = phi(errors[a], errors[b])
        shown = "n/a (no variation)" if r is None else f"{r:+.2f}"
        print(f"error correlation {a} ~ {b}: {shown}")
        if r is not None:
            pairs.append(r)
    if not pairs:
        print("No pair varies; rho cannot be estimated from these items.")
        return
    rho = max(0.0, sum(pairs) / len(pairs)) # negative average is treated as independence
    n = len(models)
    print(f"\nmean pairwise error correlation rho = {rho:.2f}")
    if rho >= 1:
        print(f"the models err together on every item: {n} models are worth one opinion")
        return
    print(f"{n} models are worth about {n_eff(n, rho):.2f} independent opinions")
    if rho > 0:
        print(f"no number of models with this rho is worth more than {1 / rho:.1f}")
    print("\nN    N_eff at this rho")
    for k in (1, 2, 3, 5, 10, 20):
        print(f"{k:<3d} {n_eff(k, rho):.2f}")

if __name__ == "__main__":
    main()

```

C5. A record of crossings, and a checker (crossings.py)

Teaches: the measurement rules of Section 9.1 as code. A record is refused if it counts an installation as a crossing, a completion without an agent-produced artefact, a stop without the person's own words, help without a description of the help, a name instead of a pseudonymous label (the label must be P followed by digits), a skipped level that is not listed exactly, a value that is not a real true or false, an unreadable date, a field the format does not know, or a level-4 crossing that names fewer than two tools asked and answered. A malformed line is reported as rejected and never stops the run. *Limits:* it checks the form of records, not their truth; whether the two tools are different model families, and whether the arrangement was rerun in a later week (a second record), are left to the researcher; it is a research instrument, not an application.

```

#!/usr/bin/env python3
"""crossings.py: a record format for Level transitions, and a checker for it.

```

Teaching example for "After Chat" (Appendix C, Listing C5). MIT licence. A research instrument, not an application: it defines what a record of one person's attempt at one transition must contain, and refuses records that would let an installation be counted as use.

The rules follow the measures published in the preregistration the paper describes (OSF, DOI 10.17605/OSF.IO/X4EGQ): a transition counts as crossed only when an artefact produced by the agent is supplied; installing is an attempt; assisted and unaided crossings are kept apart; a stop is recorded with the point where it happened and the person's own words. The record format is also published as a JSON Schema (crossings.schema.json); this checker accepts exactly what the schema accepts and adds the rules a schema cannot state.

What the checker enforces for a Level-4 crossing: at least two tools named as asked and as answered.

What it leaves to the researcher: that the tools are different model families, and that the arrangement was rerun in a later week (a second record).

Usage:

```
python3 crossings.py records.jsonl      # one JSON object per line
python3 crossings.py --demo
"""
import datetime
import json
import re
import sys

LEVELS = ("chat", "safe_agentic_environment", "agent_on_own_computer", "orchestrated_work")
ARTEFACTS = ("file", "folder", "screenshot", "pasted_output", "none")
DECISIONS = ("accept", "revise", "escalate", "discard")
REQUIRED = ("record_id", "person", "level_from", "level_to", "materials_sent", "completed", "assisted")
# Every key a record may carry; the same set as the properties of crossings.schema.json.
ALLOWED = REQUIRED + ("skipped", "intervention", "artefact", "stop_point", "stop_reason_own_words",
                     "stop_reason_code", "multi_model", "interface_version", "operating_system",
                     "network_access", "permission_mode", "policy_constraint", "acceptance_decision")
PERSON = re.compile(r"^[0-9]+$")
```

```
def parse_date(value):
    try:
        return datetime.date.fromisoformat(value)
    except (TypeError, ValueError):
        return None

def check(rec):
    problems = []
    if not isinstance(rec, dict):
        return ["record must be a JSON object"]
    for key in REQUIRED:
        if key not in rec:
            problems.append(f"missing field '{key}'")
    unknown = sorted(set(rec) - set(ALLOWED))
    if unknown:
        problems.append("unknown fields (the schema forbids them): " + ", ".join(unknown))
    if problems:
        return problems
    if not isinstance(rec["person"], str) or not PERSON.match(rec["person"]):
        problems.append("person must be a pseudonymous label such as P07, never a name")
    for key in ("completed", "assisted"):
        if not isinstance(rec[key], bool):
            problems.append(f"{key} must be true or false, not {rec[key]!r}")
    if rec["level_from"] not in LEVELS or rec["level_to"] not in LEVELS:
        problems.append("level must be one of " + ", ".join(LEVELS))
    else:
        i_from, i_to = LEVELS.index(rec["level_from"]), LEVELS.index(rec["level_to"])
        if i_to <= i_from:
            problems.append("level_to must be above level_from")
        else:
            # Levels are ordered, not compulsory: a route may go from chat straight to an agent on the
            # person's own computer. A skip is allowed, but it is recorded, never silent, and exactly.
            expected = list(LEVELS[i_from + 1:i_to])
            if expected and rec.get("skipped") != expected:
                problems.append("a move over more than one level must list exactly the levels it "
                                "skipped in 'skipped': " + ", ".join(expected))
            if not expected and rec.get("skipped"):
                problems.append("'skipped' must be empty or absent for a move of one level")
    sent = parse_date(rec["materials_sent"])
    if sent is None:
        problems.append("materials_sent must be an ISO date (YYYY-MM-DD)")
    art = rec.get("artefact", {"type": "none"})
    if not isinstance(art, dict) or art.get("type") not in ARTEFACTS:
        problems.append("artefact.type must be one of " + ", ".join(ARTEFACTS))
```

```

    art = {"type": "none"}
    if "produced_by_agent" in art and not isinstance(art["produced_by_agent"], bool):
        problems.append("artefact.produced_by_agent must be true or false")
    if "date" in art:
        art_date = parse_date(art["date"])
        if art_date is None:
            problems.append("artefact.date must be an ISO date (YYYY-MM-DD)")
        elif sent and art_date < sent:
            problems.append("artefact dated before the materials were sent")
    if rec["completed"] is True:
        # Installation is an attempt; only something the agent produced counts as a crossing.
        if art.get("type") == "none" or art.get("produced_by_agent") is not True:
            problems.append("completed=true needs an artefact with produced_by_agent=true")
        if rec["level_to"] == "orchestrated_work":
            mm = rec.get("multi_model") or {}
            asked = mm.get("asked") if isinstance(mm, dict) else None
            answered = mm.get("answered") if isinstance(mm, dict) else None
            if not isinstance(asked, list) or len(set(asked)) < 2 or \
                not isinstance(answered, list) or len(set(answered)) < 2:
                problems.append("a level-4 crossing needs multi_model.asked and multi_model.answered "
                                "with at least two tools each (Table 1: two model families)")
    elif rec["completed"] is False:
        if not rec.get("stop_point") or not rec.get("stop_reason_own_words"):
            problems.append("a non-completion records stop_point and stop_reason_own_words")
    if rec["assisted"] is True and not rec.get("intervention"):
        problems.append("assisted=true needs a description of the intervention")
    if "acceptance_decision" in rec and rec["acceptance_decision"] not in DECISIONS:
        problems.append("acceptance_decision must be one of " + ", ".join(DECISIONS))
    if "network_access" in rec and not isinstance(rec["network_access"], bool):
        problems.append("network_access must be true or false")
    return problems

```

```

DEMO = [
    {"record_id": "r1", "person": "P01", "level_from": "chat", "level_to": "safe_agentic_environment",
     "materials_sent": "2026-09-08", "completed": True, "assisted": False,
     "artefact": {"type": "folder", "date": "2026-09-09", "produced_by_agent": True}},
    {"record_id": "r2", "person": "P02", "level_from": "chat", "level_to": "safe_agentic_environment",
     "materials_sent": "2026-09-08", "completed": True, "assisted": False,
     "artefact": {"type": "screenshot", "produced_by_agent": False}},
    {"record_id": "r3", "person": "P03", "level_from": "chat", "level_to": "agent_on_own_computer",
     "skipped": ["safe_agentic_environment"], "materials_sent": "2026-09-08", "completed": False,
     "assisted": False, "stop_point": "(invented) download page",
     "stop_reason_own_words": "(invented) the page offered four versions and I did not know which"},
    {"record_id": "r4", "person": "Jane", "level_from": "chat", "level_to": "orchestrated_work",
     "materials_sent": "2026-09-08", "completed": False, "assisted": True},
]

```

```

def main():
    if len(sys.argv) > 1 and sys.argv[1] != "--demo":
        records = []
        for n, line in enumerate(open(sys.argv[1]), 1):
            if line.strip():
                try:
                    records.append(json.loads(line))
                except json.JSONDecodeError as exc:
                    records.append({"_malformed": f"line {n}: {exc}"})
    else:
        records = DEMO
        print("(demo: four invented records, two of them deliberately wrong)\n")
    bad = 0
    for rec in records:
        try:
            if isinstance(rec, dict) and "_malformed" in rec:
                problems = ["not valid JSON: " + rec["_malformed"]]
            else:
                problems = check(rec)
        except Exception as exc: # a malformed record is rejected, never a crash

```

```

        problems = [f"malformed record: {exc!r}"]
        bad += bool(problems)
        rid = rec.get("record_id", "?") if isinstance(rec, dict) else "?"
        print(f"{str(rid):6s} {'OK' if not problems else 'REJECTED'}")
        for p in problems:
            print(f"    - {p}")
        print(f"\n{len(records) - bad} of {len(records)} records accepted")
        sys.exit(1 if bad else 0)

```

```

if __name__ == "__main__":
    main()

```

The same record format is published as a JSON Schema (`crossings.schema.json`) so that other tools can validate the shape of a record; the checker above enforces the rules the schema cannot express (that an installation is not a crossing, that a stop needs the person's own words, that help needs a description, and that a skip is recorded).

```

{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://example.org/after-chat/crossing-record.schema.json",
  "title": "CrossingRecord",
  "description": "One person's attempt at one transition of the four-level model. Companion to crossings.py, which accepts exactly what this schema accepts and adds the rules the schema cannot express (installation is not a crossing; a stop needs the person's own words; help needs a description; a skip is recorded exactly; a level-4 crossing names at least two tools asked and answered). MIT licence.",
  "type": "object",
  "required": ["record_id", "person", "level_from", "level_to", "materials_sent", "completed", "assisted"],
  "properties": {
    "record_id": {"type": "string"},
    "person": {"type": "string", "pattern": "^[0-9]+$", "description": "Pseudonymous label such as P07. Never a name."},
    "level_from": {"enum": ["chat", "safe_agentic_environment", "agent_on_own_computer", "orchestrated_work"]},
    "level_to": {"enum": ["chat", "safe_agentic_environment", "agent_on_own_computer", "orchestrated_work"]},
    "skipped": {"type": "array", "items": {"enum": ["safe_agentic_environment", "agent_on_own_computer"]}, "description": "Exactly the levels passed over when the move spans more than one level. A skip is allowed and never silent."},
    "materials_sent": {"type": "string", "format": "date"},
    "completed": {"type": "boolean"},
    "assisted": {"type": "boolean"},
    "intervention": {"type": "string", "description": "Required when assisted is true: what the helper did."},
    "artefact": {
      "type": "object",
      "properties": {
        "type": {"enum": ["file", "folder", "screenshot", "pasted_output", "none"]},
        "date": {"type": "string", "format": "date"},
        "produced_by_agent": {"type": "boolean"},
        "sha256": {"type": "string", "description": "Hash of the artefact if deposited; never the content."}
      }
    },
    "required": ["type"],
    "stop_point": {"type": "string", "description": "Required when completed is false: where the attempt stopped."},
    "stop_reason_own_words": {"type": "string", "description": "Required when completed is false: the person's own words, verbatim."},
    "stop_reason_code": {"type": "string", "description": "Assigned after collection by a published coding scheme (Appendix A)."},
    "multi_model": {
      "type": "object",
      "description": "For transition 3: the denominator of a council run. A completed level-4 record needs at least two tools in asked and in answered; that they are different model families is for the researcher to verify."
    }
  }
}

```

```

    "properties": {
      "asked": {"type": "array", "items": {"type": "string"}},
      "answered": {"type": "array", "items": {"type": "string"}},
      "timed_out": {"type": "array", "items": {"type": "string"}},
      "diverged": {"type": ["boolean", "null"]},
      "changed_decision": {"type": ["boolean", "null"]},
      "coverage_hit": {"type": ["boolean", "null"]}
    },
    "interface_version": {"type": "string", "description": "Optional implementation record: the agent and its version."},
    "operating_system": {"type": "string", "description": "Optional implementation record."},
    "network_access": {"type": "boolean", "description": "Optional implementation record: whether the agent had network access."},
    "permission_mode": {"type": "string", "description": "Optional implementation record: the permission mode in force."},
    "policy_constraint": {"type": "string", "description": "Optional implementation record: any employer or platform rule that applied."},
    "acceptance_decision": {"enum": ["accept", "revise", "escalate", "discard"], "description": "Optional: the person's decision from the acceptance record."},
  },
  "additionalProperties": false
}

```

C6. Safety before level 3 (safety/)

Teaches: the practices of Section 5.3 as configuration. The first file is an example project settings file for Claude Code: plan mode by default (the vendor renamed its default mode “Manual” in July 2026; “plan” remains valid and is the safer teaching choice), the common destructive and download commands denied, the usual secret files denied both to the file reader and to a shell cat, deletions and web fetches asked about. A deny list is only a list: a permissive mode or an unlisted command can route around it, so the sandbox settings matter as much as the rules. The second is an example configuration for OpenAI’s Codex, given in its per-run, folder-scoped form: writes limited to the working folder, no network, approval on request. The third is a checklist. *Limits:* permission rules and sandboxes reduce the blast radius; they do not make an agent trustworthy, and rule syntax changes between versions.

```

{
  "permissions": {
    "defaultMode": "plan",
    "deny": [
      "Bash(rm -rf:*)",
      "Bash(sudo:*)",
      "Bash(curl:*)",
      "Bash(wget:*)",
      "Bash(cat .env:*)",
      "Bash(git push:*)",
      "Read(./env)",
      "Read(./env.*)",
      "Read(./secrets/**)"
    ],
    "ask": [
      "Bash(rm:*)",
      "Bash(mv:*)",
      "WebFetch"
    ]
  }
}

```

```

# Folder-scoped, per-run form (the policy of this paper): run the agent from the working copy with
#   codex -C <folder> -s workspace-write -a on-request

```

```

# The same values in ~/.codex/config.toml apply to every run on the machine.
# The agent may write only inside the working folder, has no network by default,
# and asks before anything outside those limits.
approval_policy = "on-request"
sandbox_mode = "workspace-write"

[sandbox_workspace_write]
network_access = false

# Pre-flight checklist before a person lets an agent act on their own computer

1. Work in a copy. Make a new folder for the agent and put copies of the files in it, never originals.
   If the files live in a cloud-synchronised tree, copy them out of it first: agents have deleted the
   zero-byte placeholders that cloud services leave for offloaded files, taking the originals with them.
2. Start in plan mode. Ask for a written plan first; read it; only then let the agent act.
3. Keep secrets out. No passwords, keys or client documents in the working folder.
4. Deny network by default. Turn it on only for a job that needs it, and then for named sites.
5. Confirm before anything irreversible: sending, deleting, publishing, purchasing.
6. Treat what the agent reads on the web as content, never as instructions. A page can carry hidden text
   written to redirect an agent.
7. Treat another agent's answer the same way: content to compare, never instructions to act on. A second
   model can pass on instructions it picked up from a page it read.
8. Use a separate browser profile for any agent that browses, signed out of banking and health sites.
9. Never type passwords, one-time codes or card numbers into the agent.
10. Ask "what could go wrong with this?" before approving a command you do not understand.
11. Keep an undo: a snapshot of the folder (or a version-control commit) before each larger change.
12. Keep a working stop. Before a long job, know how to stop it and whether stopping deletes the work.
13. Stop and revoke access when the task, the workspace or the sensitivity of the data changes.
14. Escalate a high-consequence decision to a person who is accountable for it; the agent proposes, the
    person decides.

```

C7. Supervision traces as a log (jsonl_log.py)

Teaches: the smallest useful record of a manager's work, one timestamped JSON object per event, so that plans refused, diffs rejected, second opinions taken and runs completed (the supervision traces of proposition P10) can be counted later with the standard library. Run with `--demo`. *Limits:* it is a writer, not a tracer; it records what the person or the agent tells it.

```

#!/usr/bin/env python3
"""jsonl_log.py: append one timestamped event to a JSONL file.

```

Teaching example for "After Chat" (Appendix C, Listing C7). MIT Licence.

The smallest useful record of a manager's work: one JSON object per event, one event per line, each with a UTC timestamp. Events such as "plan_refused", "diff_rejected", "second_opinion_taken" or "run_completed" are the supervision traces that proposition P10 asks for. A later paper can parse the file with the standard library. This is a writer, not a tracer: it records what you tell it.

Usage:

```

python3 jsonl_log.py run.jsonl '{"event": "plan_refused", "task": "friday-pack"}'
python3 jsonl_log.py --demo
"""

```

```

import json
import sys
from datetime import datetime, timezone
from pathlib import Path

```

```

def log(path, event):
    """Append one event; the timestamp is added unless the caller set one."""
    event = dict(event)
    event.setdefault("ts", datetime.now(timezone.utc).isoformat(timespec="seconds"))
    with Path(path).open("a", encoding="utf-8") as f:
        f.write(json.dumps(event, ensure_ascii=False) + "\n")

```

```

    return event

def main():
    if len(sys.argv) == 2 and sys.argv[1] == "--demo":
        import tempfile
        path = Path(tempfile.mkdtemp()) / "demo.jsonl"
        for ev in ({
            "event": "plan_refused", "task": "demo"},
            {"event": "second_opinion_taken", "task": "demo", "family": "other-vendor"},
            {"event": "run_completed", "task": "demo", "artefact": "demo/plan/"}):
            log(path, ev)
        print(path.read_text(), end="")
        return
    if len(sys.argv) < 3:
        print('usage: jsonl_log.py FILE \'{"event": "plan_refused"}\' or jsonl_log.py --demo',
              file=sys.stderr)
        sys.exit(2)
    print(json.dumps(log(sys.argv[1], json.loads(sys.argv[2]))))

if __name__ == "__main__":
    main()

```

Appendix D. Worked table of the effective number of opinions

A manager of agents who does not want to run listing C4 can still use the ceiling. Each cell is $N_{\text{eff}} = N / (1 + (N - 1) \rho)$; the last row is the ceiling $1/\rho$, which no number of models with that correlation can exceed. The values of ρ were chosen to illustrate the formula and measure nothing. In use, ρ is the mean pairwise correlation of errors, a different quantity from the share of joint errors on which two models agree, and it has to be estimated on items with known answers in the domain at hand (listing C4).

N models	$\rho = 0.2$ (illustration)	$\rho = 0.5$ (illustration)	$\rho = 0.6$ (illustration)	$\rho = 0.8$ (illustration)
2	1.67	1.33	1.25	1.11
3	2.14	1.50	1.36	1.15
5	2.78	1.67	1.47	1.19
10	3.57	1.82	1.56	1.22
Ceiling, $1/\rho$	5.00	2.00	1.67	1.25

The table is the reason Section 6 tells a person to spend the next subscription on a different family instead of a tenth window of the same one. Diversity of families, a claim-by-claim check against sources and a preserved dissent do the work that a larger N cannot do. Because ρ depends on the task, it should be estimated on items from the person's own domain (listing C4).

Appendix E. Landscape census, 19 to 21 September 2026

Table 5 in the main text groups tools by pattern. This appendix records the raw census behind it, for readers who want the numbers as they stood on the dates given. Star counts

move daily; they trace popularity and say nothing about quality or the number of users, so they should not be cited as of the reader's date. The pattern column is my mapping, and the projects themselves make no such claim.

Project	Location	Stars (approximate, 21 September 2026)	Licence	Pattern
anthropics/claude-code	github.com/anthropics/claude-code	147,000	Source-available, proprietary licence	Terminal agent (level 3)
openai/codex	github.com/openai/codex	126,000	Apache-2.0	Terminal agent (level 3)
google-gemini/gemini-cli	github.com/google-gemini/gemini-cli	107,000	Apache-2.0	Terminal agent; individual tiers no longer served since 18 June 2026
cline/cline	github.com/cline/cline	69,000	Apache-2.0	Editor and terminal agent
openai/codex-plugin-cc	github.com/openai/codex-plugin-cc	33,400	Apache-2.0	Official plugin: one vendor's agent inside another's
jackwener/OpenCLI	github.com/jackwener/OpenCLI	29,500	See repository	Web chat exposed to agents
karpathy/llm-council	github.com/karpathy/llm-council	24,900	See repository	Council on a programming interface
ai-shifu/ChatALL	github.com/ai-shifu/ChatALL	16,500 (about 847,000 downloads of the latest release)	Apache-2.0	Browser broadcast
BeehiveInnovations/pal-mcp-server	github.com/BeehiveInnovations/pal-mcp-server	11,800 (last commit December 2025)	Apache-2.0	Consultation server between models
MoonshotAI/kimi-cli	github.com/MoonshotAI/kimi-cli	11,400	Apache-2.0	Terminal agent (level 3)
chathub-dev/chathub	github.com/chathub-dev/chathub	10,700	GPL-3.0	Browser broadcast
nylbn/claude-octopus	github.com/nylbn/claude-octopus	4,100	MIT	CLI council
aarondfrancis/counselors	github.com/aarondfrancis/counselors	660	MIT	CLI council writing one file per tool

Project	Location	Stars (approximate, 21 September 2026)	Licence	Pattern
xai-org/grok-build	github.com/xai- org/grok-build	26,900	Apache-2.0	Terminal agent with headless mode and agent protocol